**سیستم‌عامل | زمان‌بندی پردازنده**

مقالهٔ علمی ـ آموزشی · گزارش فنی

# الگوریتم زمان‌بندی پردازنده Longest Job First (LJF)

مفاهیم، انواع، مثال‌های عددی، تحلیل ریاضی و پیاده‌سازی

پژوهشی جامع و آموزشی در حوزهٔ سیستم‌عامل و نظریهٔ زمان‌بندی

**نام دانشجو:**

صادق عضدی

**دانشگاه:**

موسسه آموزش عالی اشراق

# چکیده

این مقاله مطالعه‌ای جامع دربارهٔ الگوریتم زمان‌بندی Longest Job First (LJF) ارائه می‌کند؛ الگوریتمی که از میان فرایندهای آمادهٔ اجرا، فرایندی را انتخاب می‌کند که بیشترین زمان اجرای پردازنده (Burst Time) را دارد. پس از مرور مفاهیم فرایند، معیارهای زمان‌بندی و جایگاه LJF، نسخهٔ غیرپیش‌دستانهٔ الگوریتم تعریف می‌شود، تفاوت آن با نسخه‌های پیش‌دستانه و به‌ویژه Longest Remaining Time First (LRTF) با دقت بیان می‌شود و مراحل اجرا با فلوچارت شرح داده می‌شود. پنج مثال کاملاً حل‌شده (ورود هم‌زمان، ورود متفاوت، Burst Time یکسان، بار ده‌فرایندی با تحلیل صف آماده و مقایسهٔ پیش‌دستانه) همراه با نمودار گانت و زمان‌های انتظار، برگشت و پاسخ آمده است. تحلیل ریاضی با استدلال جابه‌جایی و راستی‌آزمایی کامل نشان می‌دهد که LJF در ورود هم‌زمان بیشترین میانگین زمان انتظار را ایجاد می‌کند، در حالی که ترتیب «بلندترین اول» برای کمینه‌سازی زمان اتمام کل روی چند پردازنده روشی ابتکاری و معتبر است. مقاله LJF را با FCFS، SJF، SRTF، Round Robin، زمان‌بندی اولویت‌دار و LRTF مقایسه می‌کند، پیاده‌سازی آزمون‌شده به زبان‌های Python، C++ و Java ارائه می‌دهد و کاربردها، محدودیت‌ها و بهبودهایی مانند Aging و روش‌های ترکیبی را بررسی می‌کند.

**کلیدواژه‌ها:** زمان‌بندی پردازنده؛ Longest Job First؛ LJF؛ LRTF؛ زمان‌بندی غیرپیش‌دستانه؛ زمان انتظار؛ زمان برگشت؛ گرسنگی؛ نمودار گانت

## Abstract

This article presents a comprehensive study of the Longest Job First (LJF) CPU scheduling algorithm, which dispatches the ready process with the largest CPU burst time. It defines the non-preemptive algorithm, distinguishes it from preemptive relatives such as Longest Remaining Time First (LRTF), and gives a flowchart and five fully solved examples with Gantt charts and waiting, turnaround and response times. An exchange argument and exhaustive verification show that LJF maximises the average waiting time for simultaneous arrivals, while longest-first ordering remains a sound heuristic for makespan on parallel machines. LJF is compared with FCFS, SJF, SRTF, Round Robin, priority scheduling and LRTF; tested Python, C++ and Java implementations, applications, limitations and improvements such as aging are presented.

**Keywords:** CPU scheduling; Longest Job First; LJF; LRTF; non-preemptive scheduling; waiting time; turnaround time; starvation; Gantt chart

# فهرست مطالب

**مقدمه** ..... **۹**

**فصل ۱: مفاهیم پایه** ..... **۹**

۱٫۱ مقدمه‌ای بر سیستم‌عامل ..... ۱۰

۱٫۲ تعریف پردازش (Process) ..... ۱۰

۱٫۳ چرخهٔ عمر پردازش ..... ۱۰

۱٫۴ تعریف زمان‌بندی پردازنده (CPU Scheduling) ..... ۱۱

۱٫۵ اهمیت زمان‌بندی پردازنده ..... ۱۱

۱٫۶ اهداف و معیارهای طراحی ..... ۱۱

۱٫۷ جایگاه LJF در میان الگوریتم‌های زمان‌بندی ..... ۱۱

**فصل ۲: آشنایی کامل با الگوریتم LJF** ..... **۱۲**

۲٫۱ تعریف دقیق Longest Job First ..... ۱۲

۲٫۲ تاریخچه و زمینهٔ استفاده ..... ۱۳

۲٫۳ ایدهٔ اصلی و منطق عملکرد ..... ۱۳

۲٫۴ نحوهٔ انتخاب فرایند بعدی ..... ۱۳

۲٫۵ شرایط ورود فرایندها به صف آماده ..... ۱۳

۲٫۶ نحوهٔ مدیریت صف فرایندها ..... ۱۴

۲٫۷ نقش Burst Time ..... ۱۴

۲٫۸ تفاوت زمان ورود و زمان اجرای پردازش ..... ۱۴

۲٫۹ بررسی رفتار الگوریتم در شرایط مختلف ..... ۱۴

**فصل ۳: انواع الگوریتم LJF** ..... **۱۵**

۳٫۱ Non-Preemptive LJF ..... ۱۵

۳٫۲ Preemptive LJF ..... ۱۵

۳٫۳ بررسی مفهوم پیش‌دستی (Preemption) ..... ۱۵

۳٫۴ تفاوت نسخهٔ پیش‌دستانه و غیرپیش‌دستانه ..... ۱۶

۳٫۵ شرایط تغییر پردازش در حال اجرا ..... ۱۶

۳٫۶ مزایا و معایب هر نسخه ..... ۱۶

۳٫۷ مثال‌های مستقل برای هر نسخه ..... ۱۷

۳٫۸ اصطلاحات مرتبط: تفاوت LJF با LRTF ..... ۱۷

**فصل ۴: نحوهٔ عملکرد و مراحل اجرایی** ..... **۱۷**

۴٫۱ رویهٔ گام‌به‌گام ..... ۱۷

۴٫۲ فلوچارت ..... ۱۸

۴٫۳ شبه‌کد ..... ۱۸

**فصل ۵: مثال‌های عددی و حل گام‌به‌گام** ..... **۱۹**

۵٫۱ مثال ۱: ورود هم‌زمان همهٔ فرایندها در زمان ۰ ..... ۱۹

۵٫۲ مثال ۲: زمان‌های ورود متفاوت ..... ۲۰

۵٫۳ مثال ۳: Burst Time یکسان ..... ۲۲

۵٫۴ مثال ۴: ده فرایند و تحلیل صف آماده ..... ۲۴

۵٫۵ مثال ۵: نسخهٔ پیش‌دستانه (LRTF) در برابر LJF غیرپیش‌دستانه ..... ۲۶

**فصل ۶: فرمول‌ها و تحلیل ریاضی** ..... **۲۸**

۶٫۱ زمان برگشت (Turnaround Time) ..... ۲۸
۶٫۲ زمان انتظار (Waiting Time) ..... ۲۸
۶٫۳ میانگین زمان انتظار ..... ۲۸
۶٫۴ میانگین زمان برگشت ..... ۲۹
۶٫۵ زمان پاسخ (Response Time) ..... ۲۹
۶٫۶ بهره‌وری CPU و گذردهی ..... ۲۹
۶٫۷ تحلیل بهینگی: چرا LJF میانگین زمان انتظار را بیشینه می‌کند ..... ۳۰
۶٫۸ جایی که «بلندترین اول» خوب است: زمان اتمام کل روی ماشین‌های موازی ..... ۳۱
۶٫۹ پیچیدگی زمانی و فضایی ..... ۳۱
**فصل ۷: نمودارها و تصاویر آموزشی ..... ۳۲**
**فصل ۸: مزایا و معایب الگوریتم LJF ..... ۳۵**
۸٫۱ مزایا ..... ۳۵
۸٫۲ معایب ..... ۳۵
**فصل ۹: مقایسه با الگوریتم‌های دیگر ..... ۳۷**
۹٫۱ مقایسهٔ کیفی ..... ۳۷
۹٫۲ رفتار در شرایط مختلف ..... ۳۷
۹٫۳ مقایسهٔ عددی ..... ۳۸
**فصل ۱۰: پیاده‌سازی برنامه‌نویسی ..... ۳۸**
۱۰٫۱ Python ..... ۳۹
۱۰٫۲ C++ ..... ۴۰
۱۰٫۳ Java ..... ۴۲
۱۰٫۴ ورودی و خروجی نمونه ..... ۴۳
۱۰٫۵ حالت‌های مرزی و بررسی صحت نتایج ..... ۴۴
**فصل ۱۱: مثال‌های کاربردی و مطالعات موردی ..... ۴۵**
۱۱٫۱ کاربرد نظری در برابر استفادهٔ عملی در سیستم‌عامل‌های واقعی ..... ۴۵
۱۱٫۲ پردازش دسته‌ای و سیستم‌های آفلاین ..... ۴۵
۱۱٫۳ محیط‌های شبیه‌سازی آموزشی ..... ۴۶
۱۱٫۴ محیط‌های محاسباتی با وظایف طولانی ..... ۴۶
**فصل ۱۲: بهبودها و روش‌های جایگزین ..... ۴۶**
۱۲٫۱ Aging ..... ۴۷
۱۲٫۲ Starvation Prevention ..... ۴۷
۱۲٫۳ Hybrid Scheduling ..... ۴۷
۱۲٫۴ Dynamic Priority ..... ۴۷
۱۲٫۵ ترکیب LJF و Priority Scheduling ..... ۴۷
۱۲٫۶ مدیریت بهتر پردازش‌های کوتاه ..... ۴۸
۱۲٫۷ سیاست‌های جایگزین برای افزایش عدالت ..... ۴۸
**فصل ۱۳: جمع‌بندی و نتیجه‌گیری ..... ۴۸**
۱۳٫۱ مرور مفاهیم اصلی ..... ۴۸
۱۳٫۲ خلاصهٔ عملکرد الگوریتم و نتایج تحلیل عددی ..... ۴۹
۱۳٫۳ ارزیابی نقاط قوت و ضعف ..... ۴۹
۱۳٫۴ مقایسه با الگوریتم‌های دیگر ..... ۴۹
۱۳٫۵ محدودیت‌های الگوریتم و این مطالعه ..... ۴۹

۱۳٫۶ پیشنهاد برای تحقیقات آینده ..... ۴۹
**فصل ۱۴: منابع و مراجع علمی ..... ۵۰**

# فهرست جداول

جدول ۱ـ مفاهیم پایهٔ مورد استفاده در مقاله ..... ۱۲
جدول ۲ـ مقایسهٔ LJF غیرپیش‌دستانه، LJF-PA و LRTF ..... ۱۶
جدول ۳ـ نتایج سه نسخه روی بار کاری مثال ۵ ..... ۱۷
جدول ۴ـ مشخصات فرایندهای مثال ۱ ..... ۱۹
جدول ۵ـ نتایج مثال ۱ تحت LJF ..... ۲۰
جدول ۶ـ مشخصات فرایندهای مثال ۲ ..... ۲۰
جدول ۷ـ محاسبهٔ زمان اتمام در مثال ۲ ..... ۲۱
جدول ۸ـ محاسبهٔ زمان برگشت در مثال ۲ ..... ۲۱
جدول ۹ـ محاسبهٔ زمان انتظار در مثال ۲ ..... ۲۲
جدول ۱۰ـ خلاصهٔ نتایج مثال ۲ ..... ۲۲
جدول ۱۱ـ مشخصات فرایندهای مثال ۳ ..... ۲۲
جدول ۱۲ـ نتایج مثال ۳ تحت LJF ..... ۲۳
جدول ۱۳ـ مشخصات فرایندهای مثال ۴ ..... ۲۴
جدول ۱۴ـ صف آماده در هر لحظهٔ تصمیم در مثال ۴ (مرتب بر حسب BT نزولی) ..... ۲۵
جدول ۱۵ـ نتایج مثال ۴ تحت LJF ..... ۲۶
جدول ۱۶ـ مشخصات فرایندهای مثال ۵ ..... ۲۶
جدول ۱۷ـ نتایج مثال ۵: LJF (غیرپیش‌دستانه) ..... ۲۷
جدول ۱۸ـ نتایج مثال ۵: LRTF (پیش‌دستانه) ..... ۲۷
جدول ۱۹ـ خلاصهٔ فرمول‌ها ..... ۳۰
جدول ۲۰ـ پیچیدگی زمانی و فضایی پیاده‌سازی‌های LJF ..... ۳۱
جدول ۲۱ـ فهرست شکل‌ها ..... ۳۲
جدول ۲۲ـ مزایا و معایب LJF ..... ۳۶
جدول ۲۳ـ مقایسهٔ جامع الگوریتم‌های زمان‌بندی ..... ۳۷
جدول ۲۴ـ میانگین معیارهای همهٔ الگوریتم‌ها روی بار کاری مثال ۲ ..... ۳۸
جدول ۲۵ـ توضیح خط‌به‌خط برنامهٔ Python ..... ۴۰
جدول ۲۶ـ توضیح خط‌به‌خط برنامهٔ C++ ..... ۴۱
جدول ۲۷ـ توضیح خط‌به‌خط برنامهٔ Java ..... ۴۳
جدول ۲۸ـ راستی‌آزمایی پیاده‌سازی‌ها روی بارهای کاری مقاله ..... ۴۴
جدول ۲۹ـ مناسب بودن LJF در محیط‌های مختلف ..... ۴۶
جدول ۳۰ـ روش‌های مکمل یا جایگزین LJF ..... ۴۸
جدول ۳۱ـ خلاصهٔ نتایج عددی (LJF مگر خلاف آن گفته شود، همراه با AWT الگوریتم SJF برای مرجع) ..... ۴۹

# فهرست اشکال

شکل ۱ـ نمودار حالات فرایند؛ تصمیم LJF روی یال توزیع (آماده ← در حال اجرا) گرفته می‌شود. ..... ۱۰
شکل ۲ـ فلوچارت LJF غیرپیش‌دستانه همراه با شاخهٔ بیکاری CPU. ..... ۱۸
شکل ۳ـ نمودار گانت مثال ۱. ..... ۲۰
شکل ۴ـ نمودار گانت مثال ۲ (پیکان‌ها ورودها را نشان می‌دهند). ..... ۲۱
شکل ۵ـ نمودار گانت مثال ۳ (BT یکسان). ..... ۲۳
شکل ۶ـ نمودار گانت مثال ۴. ..... ۲۵

شکل ۷ـ مثال ۵: LJF غیرپیش‌دستانه (بالا) در برابر LRTF (پایین). ..... ۲۷
شکل ۸ـ مجموع زمان انتظار روی هر ۱۲۰ ترتیب ممکن بار کاری مثال ۱. ..... ۳۰
شکل ۹ـ رشد هزینهٔ کل انتخاب: پویش خطی ($n^2$) در برابر هیپ ($n \log n$). ..... ۳۲
شکل ۱۰ـ ترتیب اجرای بار کاری مثال ۱ تحت FCFS، SJF و LJF. ..... ۳۳
شکل ۱۱ـ طول صف آماده در لحظات تصمیم مثال ۴ ..... ۳۳
شکل ۱۲ـ زمان انتظار هر فرایند مثال ۱ تحت FCFS، SJF و LJF. ..... ۳۳
شکل ۱۳ـ زمان برگشت هر فرایند مثال ۱ تحت FCFS، SJF و LJF. ..... ۳۴
شکل ۱۴ـ زمان انتظار مثال ۲ تحت LJF و FCFS. ..... ۳۴
شکل ۱۵ـ زمان انتظار مثال ۲ تحت LJF و SJF. ..... ۳۴

# فهرست اختصارات

| اختصار | معنا |
|---|---|
| AT | Arrival Time (زمان ورود) |
| ATAT | Average Turnaround Time (میانگین زمان برگشت) |
| AWT | Average Waiting Time (میانگین زمان انتظار) |
| ART | Average Response Time (میانگین زمان پاسخ) |
| BT | Burst Time (زمان اجرا) |
| CPU | Central Processing Unit (پردازندهٔ مرکزی) |
| CT | Completion Time (زمان اتمام) |
| FCFS | First Come First Served |
| HRRN | Highest Response Ratio Next |
| I/O | ورودی / خروجی |
| LJF | Longest Job First |
| LJF-PA | LJF پیش‌دستانه در لحظهٔ ورود (کلید ایستا) |
| LPT | Longest Processing Time (قاعده) |
| LRTF | Longest Remaining Time First |
| NP | Non-Preemptive (غیرپیش‌دستانه) |
| OS | Operating System (سیستم‌عامل) |
| PCB | Process Control Block |
| PID | Process Identifier (شناسهٔ فرایند) |
| RR | Round Robin |
| RT | Response Time (زمان پاسخ) |
| SJF | Shortest Job First |
| SRTF | Shortest Remaining Time First |
| ST | Start Time (اولین تخصیص CPU) |
| TAT | Turnaround Time (زمان برگشت) |
| WT | Waiting Time (زمان انتظار) |

# مقدمه

پردازندهٔ مرکزی (CPU) کمیاب‌ترین منبع در یک رایانهٔ چندبرنامگی است: تا وقتی یک فرایند آن را در اختیار دارد، سایر فرایندهای آمادهٔ اجرا باید منتظر بمانند. بخشی از سیستم‌عامل که تعیین می‌کند کدام فرایند آماده در نوبت بعد پردازنده را بگیرد، زمان‌بند پردازنده (CPU Scheduler) و قاعدهٔ تصمیم‌گیری آن، الگوریتم زمان‌بندی نام دارد. این قاعده تعیین می‌کند کاربران چقدر منتظر می‌مانند، سخت‌افزار چقدر مشغول است و کار چقدر منصفانه تقسیم می‌شود؛ از این رو زمان‌بندی از پرتحلیل‌ترین موضوعات سیستم‌عامل است [۱, ۲, ۳, ۹].

این مقاله یکی از این قواعد، یعنی Longest Job First (LJF) را بررسی می‌کند. LJF همیشه فرایند آماده‌ای را اجرا می‌کند که بیشترین زمان اجرا (Burst Time) را دارد. این الگوریتم تصویر آینه‌ای Shortest Job First (SJF) است و همین ویژگی آن را به ابزاری آموزشی ارزشمند تبدیل می‌کند: مقایسهٔ این دو با محاسبات دقیق نشان می‌دهد کلید انتخاب چگونه زمان انتظار، عدالت و گرسنگی را تعیین می‌کند. در نظریهٔ زمان‌بندی نیز همین ایدهٔ «بلندترین اول» با نام قاعدهٔ Longest Processing Time (LPT) برای توازن بار روی ماشین‌های مشابه ظاهر می‌شود [۶].

مقاله در چهارده فصل سازمان یافته است. فصل‌های ۱ تا ۴ مبانی مفهومی و رویه‌ای را می‌سازند؛ فصل‌های ۵ تا ۷ پنج مثال کاملاً حل‌شده، تحلیل ریاضی و نمودارها را ارائه می‌دهند؛ فصل‌های ۸ و ۹ نقاط قوت و ضعف و مقایسه با سایر الگوریتم‌ها را بررسی می‌کنند؛ فصل‌های ۱۰ و ۱۱ پیاده‌سازی و کاربردها را دربر می‌گیرند؛ و فصل‌های ۱۲ تا ۱۴ بهبودها، نتیجه‌گیری و منابع‌اند. همهٔ نتایج عددی با یک شبیه‌ساز مرجع تولید و با سه پیاده‌سازی مستقل (Python، C++ و Java) راستی‌آزمایی شده‌اند، بنابراین جدول‌ها، نمودارهای گانت و خروجی برنامه‌ها کاملاً هم‌خوان‌اند.

# فصل ۱: مفاهیم پایه

این فصل واژگان مورد استفاده در مقاله را مشخص می‌کند: سیستم‌عامل چه می‌کند، فرایند چیست، چگونه میان حالت‌ها جابه‌جا می‌شود و زمان‌بند پردازنده چه چیزی را باید بهینه کند.

## ۱٫۱ مقدمه‌ای بر سیستم‌عامل

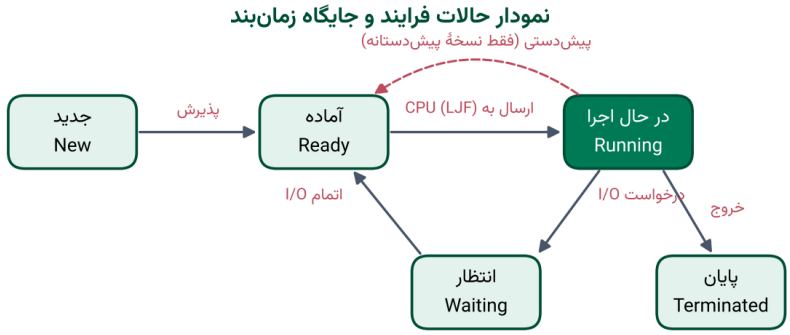
سیستم‌عامل (Operating System) نرم‌افزار سیستمی است که منابع سخت‌افزاری را مدیریت می‌کند و خدماتی در اختیار برنامه‌های کاربردی قرار می‌دهد. وظایف اصلی آن مدیریت فرایند، حافظه، فایل و ذخیره‌سازی، دستگاه‌ها و حفاظت است. زمان‌بندی پردازنده بخشی از مدیریت فرایند است: سیستم‌عامل تعداد اندکی پردازنده را میان تعداد بسیار بیشتری محاسبه تسهیم می‌کند تا هر یک پیشرفت کند [۱].

## ۱٫۲ تعریف پردازش (Process)

فرایند (Process) برنامه‌ای در حال اجراست. هر فرایند شامل کد برنامه، فعالیت جاری (شمارندهٔ برنامه و ثبات‌ها)، پشته، بخش داده و هیپ است. برنامه موجودیتی ایستا روی دیسک است، اما فرایند موجودیتی فعال با حالت است. سیستم‌عامل برای هر فرایند یک بلوک کنترل فرایند (Process Control Block یا PCB) نگه می‌دارد که شناسه، حالت، محتوای ثبات‌ها، اطلاعات زمان‌بندی و حسابداری را ذخیره می‌کند [۱].

## ۱٫۳ چرخهٔ عمر پردازش

فرایند در طول عمر خود میان پنج حالت جابه‌جا می‌شود: جدید (New)، آماده (Ready)، در حال اجرا (Running)، انتظار (Waiting) و پایان (Terminated). زمان‌بند فقط به یک گذار توجه دارد: آماده ← در حال اجرا که توزیع (Dispatch) نام دارد. شکل ۱ حالت‌ها و گذارها را نشان می‌دهد، از جمله یال اضافی «در حال اجرا ← آماده» که فقط در سیاست‌های پیش‌دستانه وجود دارد.




**شکل ۱ـ** نمودار حالات فرایند؛ تصمیم LJF روی یال توزیع (آماده ← در حال اجرا) گرفته می‌شود.

## ۱٫۴ تعریف زمان‌بندی پردازنده (CPU Scheduling)

زمان‌بندی پردازنده انتخاب فرایندی از صف آماده است که پردازنده به آن تخصیص می‌یابد. فرایندها میان دورهٔ محاسبه یا CPU Burst و دورهٔ انتظار برای دستگاه‌ها یا I/O Burst متناوب‌اند. طول CPU Burst بعدی همان زمان اجرا (Burst Time یا BT) است که LJF از آن استفاده می‌کند. زمان‌بند کوتاه‌مدت فرایند را انتخاب می‌کند و توزیع‌کننده (Dispatcher) تعویض زمینه (Context Switch) را انجام می‌دهد؛ یعنی وضعیت فرایند خارج‌شونده را ذخیره و وضعیت فرایند واردشونده را بازیابی می‌کند [۱, ۳].

## ۱٫۵ اهمیت زمان‌بندی پردازنده

یک مجموعهٔ ثابت از فرایندها فقط با تغییر ترتیب اجرا می‌تواند زمان‌های انتظار بسیار متفاوتی داشته باشد. بخش ۶٫۷ ثابت می‌کند برای دسته‌ای از کارهای هم‌زمان‌وارد، ترتیب به‌تنهایی می‌تواند میانگین زمان انتظار را بیش از ۱٫۷ برابر تغییر دهد (مثال ۱: ۱۶٫۲ در برابر ۹٫۴ واحد). بنابراین زمان‌بندی بر پاسخ‌گویی، بهره‌وری سخت‌افزار، گذردهی و عدالت اثر مستقیم دارد.

## ۱٫۶ اهداف و معیارهای طراحی

زمان‌بندها با معیارهای قابل اندازه‌گیری ارزیابی می‌شوند که در فصل ۶ صورت‌بندی شده‌اند. جهت بهینه‌سازی هر معیار در زیر آمده است.

- بهره‌وری CPU (CPU Utilization): پردازنده تا حد ممکن مشغول بماند (بیشینه).
- گذردهی (Throughput): تعداد فرایندهای تکمیل‌شده در واحد زمان (بیشینه).
- زمان برگشت (Turnaround Time): از ورود تا اتمام (کمینه).
- زمان انتظار (Waiting Time): مجموع زمان حضور در صف آماده (کمینه).
- زمان پاسخ (Response Time): از ورود تا اولین تخصیص CPU (کمینه).
- عدالت و نبود گرسنگی: هر فرایند سرانجام اجرا شود.

## ۱٫۷ جایگاه LJF در میان الگوریتم‌های زمان‌بندی

الگوریتم‌های زمان‌بندی را می‌توان بر دو محور دسته‌بندی کرد: آیا می‌توانند پردازنده را از فرایند در حال اجرا بگیرند (پیش‌دستانه یا غیرپیش‌دستانه) و برای رتبه‌بندی فرایندهای آماده از چه کلیدی استفاده می‌کنند (ترتیب ورود، طول کار، اولویت، زمان سپری‌شده). FCFS بر اساس ورود، SJF و

SRTF بر اساس کوتاهی، Round Robin بر اساس چرخش و زمان‌بندی اولویت‌دار بر اساس عدد اولویت رتبه می‌دهند. LJF بر اساس بلندی طول کار رتبه می‌دهد و در گوشهٔ مقابل SJF قرار می‌گیرد. این سیاست بیشتر نظری و آموزشی است تا یک سیاست عمومی رایج (فصل ۱۱).

**جدول ۱ـ** مفاهیم پایهٔ مورد استفاده در مقاله

| مفهوم | تعریف | نقش در زمان‌بندی |
|---|---|---|
| Process | برنامهٔ در حال اجرا با حالت مستقل | واحدی که برای CPU رقابت می‌کند |
| PCB | رکورد هسته دربارهٔ فرایند | ذخیرهٔ حالت و اطلاعات زمان‌بندی |
| CPU Burst | بازهٔ محاسبهٔ پیوسته | طول آن (BT) کلید انتخاب LJF است |
| I/O Burst | بازهٔ انتظار برای دستگاه | فرایند را به حالت انتظار می‌برد |
| Ready Queue | مجموعهٔ فرایندهای آمادهٔ اجرا | نامزدهای توزیع بعدی |
| Dispatcher | انجام‌دهندهٔ تعویض زمینه | CPU را به فرایند منتخب می‌دهد |
| Context Switch | ذخیره/بازیابی وضعیت CPU | سربار محض؛ در پیش‌دستی زیاد است |
| Preemption | گرفتن اجباری CPU از فرایند در حال اجرا | تمایز LJF و LRTF |

**جمع‌بندی فصل**

تنها وظیفهٔ زمان‌بند تصمیم آماده ← در حال اجراست. LJF این تصمیم را بر اساس بیشترین BT می‌گیرد و بهتر است آن را قرینهٔ SJF دانست.

# فصل ۲: آشنایی کامل با الگوریتم LJF

این فصل Longest Job First را دقیق تعریف می‌کند و اطلاعاتی که مصرف می‌کند، صفی که مدیریت می‌کند و رفتار آن در الگوهای مختلف ورود را بررسی می‌کند.

## ۲٫۱ تعریف دقیق Longest Job First

Longest Job First سیاست زمان‌بندی است که هر وقت CPU آزاد شود، از صف آماده فرایندی را با بیشترین زمان اجرا برمی‌گزیند. در شکل کلاسیک (غیرپیش‌دستانه) فرایند منتخب تا پایان اجرا می‌شود. به‌صورت صوری، اگر R(t) مجموعهٔ فرایندهای واردشده تا لحظهٔ t و اجرانشده باشد، فرایند بعدی p* = argmax BT(p), p in R(t) است.

## ۲٫۲ تاریخچه و زمینهٔ استفاده

مقالهٔ مرجع واحدی وجود ندارد که LJF را به‌عنوان الگوریتم سیستم‌عامل معرفی کرده باشد؛ این سیاست عمدتاً در مواد درسی به‌عنوان نقطهٔ مقابل SJF مطرح می‌شود. تبار نظری آن قاعدهٔ LPT در نظریهٔ زمان‌بندی ماشین است که گراهام تضمین تقریب آن را برای کمینه‌سازی زمان پایان روی ماشین‌های موازی مشابه ثابت کرد [۶, ۷]. بهینگی قاعدهٔ مقابل، یعنی «کوتاه‌ترین اول»، برای میانگین زمان اتمام روی یک ماشین به اسمیت بازمی‌گردد [۵]. پس فهم LJF یعنی دانستن اینکه ترتیب بلندترین‌اول به کدام هدف خدمت می‌کند (پایان متوازن روی چند ماشین) و به کدام آسیب می‌زند (میانگین زمان انتظار روی یک ماشین).

## ۲٫۳ ایدهٔ اصلی و منطق عملکرد

این قاعده به بزرگ‌ترین کار اولویت می‌دهد تا محاسبات بزرگ زود آغاز شوند. روی یک CPU این کار برنامه را کوتاه‌تر نمی‌کند، زیرا مجموع زمان مشغولی برابر مجموع همهٔ BTهاست و ترتیب فقط انتظار را بازتوزیع می‌کند و هزینه را بر دوش فرایندهای کوتاه می‌گذارد. اما روی چند پردازنده، آغاز زودهنگام کارهای بلند از تنها ماندن یک کار بلند در پایان جلوگیری می‌کند و دلیل خوب‌بودن LPT برای توازن بار همین است.

## ۲٫۴ نحوهٔ انتخاب فرایند بعدی

در هر لحظهٔ تصمیم، زمان‌بند (۱) همهٔ فرایندهای ناتمام با AT <= t را گردآوری می‌کند، (۲) بیشترین BT را برمی‌گزیند و (۳) تساوی را به‌صورت قطعی می‌شکند. قاعدهٔ شکستن تساوی در این مقاله: ورود زودتر، سپس شمارهٔ فرایند کوچک‌تر. صریح‌بودن این قاعده مهم است، زیرا مثال ۳ چند BT برابر دارد و قواعد مختلف زمان‌بندی‌های متفاوت اما معتبر می‌دهند.

## ۲٫۵ شرایط ورود فرایندها به صف آماده

فرایند در زمان ورود (AT) وارد صف آماده می‌شود. فرایندی که هنوز نرسیده برای زمان‌بند نامرئی است، حتی اگر بزرگ‌ترین BT کل بار کاری را داشته باشد. این ویژگی که LJF فقط از میان آنچه اکنون حاضر است انتخاب می‌کند، منشأ پدیدهٔ کاروانی بخش ۲٫۹ است.

## ۲٫۶ نحوهٔ مدیریت صف فرایندها

از نظر مفهومی صف آماده بر حسب BT نزولی مرتب است. پیاده‌سازی می‌تواند فهرست نامرتب را پویش کند، فهرست مرتب نگه دارد یا از هیپ بیشینه با کلید BT استفاده کند؛ این انتخاب‌ها زمان اجرا را تغییر می‌دهند (بخش ۶٫۹) اما هرگز برنامه را عوض نمی‌کنند. اگر هیچ فرایندی آماده نباشد، CPU تا ورود بعدی بیکار می‌ماند.

## ۲٫۷ نقش Burst Time

BT تنها کمیتی است که مقایسه می‌شود. الگوریتم آن را پیش از اجرا لازم دارد، ولی سیستم‌عامل واقعی آینده را نمی‌داند و باید BT را پیش‌بینی کند، معمولاً با میانگین‌گیری نمایی از انفجارهای قبلی: tau(n+۱) = alpha t(n) + (۱ − alpha) tau(n) با ۰ <= alpha <= ۱ [۱]. پس LJF مسئلهٔ پیش‌بینی SJF را به ارث می‌برد و خطای پیش‌بینی می‌تواند ترتیب مطلوب را وارونه کند.

## ۲٫۸ تفاوت زمان ورود و زمان اجرای پردازش

زمان ورود می‌گوید فرایند چه وقت واجد شرایط می‌شود و زمان اجرا می‌گوید چه مدت CPU لازم دارد. این دو ویژگی مستقل‌اند: AT فقط شرط واجدشرایطی است و BT رتبه را تعیین می‌کند. فرایندی با BT بسیار بزرگ که دیر برسد، در نسخهٔ غیرپیش‌دستانه نمی‌تواند از کار در حال اجرا جلو بزند.

## ۲٫۹ بررسی رفتار الگوریتم در شرایط مختلف

- ورود هم‌زمان: LJF به مرتب‌سازی نزولی بر اساس BT تبدیل می‌شود (مثال ۱).
- ورود متفاوت: کارهای کوتاه اولیه ممکن است صرفاً چون چیز بلندتری حاضر نیست اجرا شوند؛ ورودهای بعدی صف را بازچینی می‌کنند (مثال ۲).
- BT برابر: نتیجه را قاعدهٔ شکستن تساوی تعیین می‌کند و با قاعدهٔ «ورود زودتر» میان فرایندهای مساوی همان FCFS است (مثال ۳).
- فاصلهٔ بیکاری: اگر صف خالی شود CPU بیکار می‌ماند و بهره‌وری زیر ۱۰۰٪ می‌رود.
- ورود مداوم کارهای بلند: کارهای کوتاه می‌توانند بی‌نهایت منتظر بمانند، یعنی گرسنگی (بخش ۸٫۲).
- یک کار بسیار بلند: در صورت حضور اول سرویس می‌گیرد و بقیه پشت آن منتظر می‌مانند.

**جمع‌بندی فصل**

LJF بیشینهٔ BT را از میان فرایندهای واردشده و ناتمام برمی‌گزیند، تساوی را با ورود زودتر می‌شکند و BT را از پیش لازم دارد. رفتار آن از یک مرتب‌سازی ساده (ورود مشترک) تا گرسنگی (ورود مداوم کارهای بلند) متغیر است.

# فصل ۳: انواع الگوریتم LJF

LJF در شکل غیرپیش‌دستانه و در خویشاوندان پیش‌دستانه وجود دارد. اصطلاحات به‌آسانی با هم خلط می‌شوند، بنابراین این فصل هر نسخه را با دقت تعریف می‌کند.

## ۳٫۱ Non-Preemptive LJF

فرایند به‌محض دریافت CPU آن را تا پایان نگه می‌دارد. تصمیم‌ها فقط در لحظات اتمام (و پس از بیکاری) گرفته می‌شود. این همان الگوریتمی است که در ادامهٔ مقاله به‌طور ساده LJF نامیده می‌شود و در فصل ۱۰ پیاده‌سازی شده است.

## ۳٫۲ Preemptive LJF

سیاست پیش‌دستانه می‌تواند فرایند در حال اجرا را پیش از پایان متوقف کند. برای قاعدهٔ «بلندترین اول» دو تعریف متمایز وجود دارد و منابع نام را آزادانه به کار می‌برند:

- LJF پیش‌دستانه با کلید ایستا (نام پیشنهادی ما: LJF-PA، پیش‌دستی هنگام ورود). کلید رتبه‌بندی همان BT اولیه می‌ماند. ورود جدید تنها وقتی فرایند در حال اجرا را پس می‌زند که BT اولیهٔ آن بزرگ‌تر از BT اولیهٔ فرایند در حال اجرا باشد.
- LJF پیش‌دستانه با کلید پویا که در همه‌جا Longest Remaining Time First (LRTF) نامیده می‌شود. کلید همان زمان باقی‌ماندهٔ اجراست که در حین اجرا کاهش می‌یابد؛ پس فرایند در حال اجرا حتی بدون ورود جدید هم می‌تواند CPU را از دست بدهد.

## ۳٫۳ بررسی مفهوم پیش‌دستی (Preemption)

پیش‌دستی گرفتن غیرارادی CPU از فرایند در حال اجراست که با تایمر یا با ورود فرایندی با رتبهٔ بهتر رخ می‌دهد. این کار زمان‌بند را واکنش‌گراتر می‌کند ولی سربار تعویض زمینه می‌افزاید و نیازمند ذخیرهٔ حالت هر فرایند قطع‌شده است [۱].

## ۳٫۴ تفاوت نسخهٔ پیش‌دستانه و غیرپیش‌دستانه

**جدول ۲.** مقایسهٔ LJF غیرپیش‌دستانه، LJF-PA و LRTF

| جنبه | LJF (غیرپیش‌دستانه) | LJF-PA | LRTF |
|---|---|---|---|
| کلید رتبه‌بندی | BT اولیه | BT اولیه | BT باقی‌مانده |
| لحظات تصمیم | اتمام‌ها | اتمام‌ها و ورودها | هر واحد زمانی |
| قطع فرایند در حال اجرا | خیر | فقط با ورود کار بزرگ‌تر | بله، حتی بدون ورود |
| تعویض زمینه | کمترین | کم | زیاد |
| همسان‌سازی زمان‌های پایان | خیر | خیر | بله (فرایندها به زمان باقی‌مانده یکسان همگرا می‌شوند) |
| پیاده‌سازی | ساده | متوسط | نیازمند تایمر یا شبیه‌سازی رویدادی |

## ۳٫۵ شرایط تغییر پردازش در حال اجرا

در LJF غیرپیش‌دستانه فرایند در حال اجرا فقط با پایان‌یافتن تغییر می‌کند. در LJF-PA با ورود فرایندی تغییر می‌کند که BT اولیه‌اش از فرایند در حال اجرا بیشتر باشد. در LRTF هرگاه فرایند آماده‌ای زمان باقی‌ماندهٔ اکیداً بزرگ‌تری از فرایند در حال اجرا داشته باشد تغییر رخ می‌دهد؛ این می‌تواند هنگام ورود یا وقتی رخ دهد که زمان باقی‌ماندهٔ فرایند در حال اجرا آن‌قدر کم شده که از دیگری کمتر شود.

## ۳٫۶ مزایا و معایب هر نسخه

LJF غیرپیش‌دستانه ساده است و کمترین سربار را دارد، اما کار بلندِ دیرآمده باید منتظر کار جاری بماند. LJF-PA به کار بلند اجازهٔ آغاز فوری می‌دهد ولی ممکن است کاری نیمه‌تمام را مدتی طولانی رها کند. LRTF بهترین واکنش به بار فعلی را دارد و همهٔ کارها را نزدیک پایان تمام می‌کند که برای کارهای دسته‌ای با هدف زمان اتمام کل مناسب است، اما تعویض‌های پیوسته‌اش بیشترین سربار و در حالت حدی تسهیم پردازنده میان کارهای مساوی را پدید می‌آورد. همهٔ نسخه‌ها ضعف LJF دربارهٔ کارهای کوتاه را دارند.

## ۳٫۷ مثال‌های مستقل برای هر نسخه

بار کاری مثال ۵ (بخش ۵٫۵) را در نظر بگیرید: P1 با (AT 0, BT 4)، P2 با (۱, ۷)، P3 با (۲, ۳) و P4 با (۴, ۶). LJF غیرپیش‌دستانه برنامهٔ [۱۷–۲۰] P3 | [۱۱–۱۷] P4 | [۴–۱۱] P2 | [۰–۴] P1 را می‌دهد.

LJF-PA فرایند P1 را فقط تا ورود P2 (BT برابر ۷ بزرگ‌تر از ۴) اجرا می‌کند و برنامهٔ P2 | [۰–۱] P1 [۱–۴] | P4 [۴–۷] | P2 [۷–۹] | P1 [۹–۱۰] | P3 [۱۰–۱۱] | P4 [۱۱–۱۳] | P1 [۱۳–۱۴] | P2 [۱۴–۱۵] | P4 [۱۵–۱۷] | P1 [۱۷–۱۸] | P2 [۱۸–۱۹] | P4 [۱۹–۲۰] را تولید می‌کند.

**جدول ۳ـ** نتایج سه نسخه روی بار کاری مثال ۵

| نسخه | AWT | ATAT | ART | تعداد پیش‌دستی |
|---|---|---|---|---|
| LJF (غیرپیش‌دستانه) | ۶٫۲۵ | ۱۱٫۲۵ | ۶٫۲۵ | ۰ |
| LJF-PA | ۸٫۰۰ | ۱۳٫۰۰ | ۴٫۷۵ | ۱ |
| LRTF | ۱۱٫۷۵ | ۱۶٫۷۵ | ۲٫۰۰ | ۹ |

## ۳٫۸ اصطلاحات مرتبط: تفاوت LJF با LRTF

**LJF و LRTF یک الگوریتم نیستند**

LJF بر اساس کل زمان اجرا رتبه می‌دهد و در شکل کلاسیک هرگز فرایند در حال اجرا را قطع نمی‌کند. LRTF بر اساس زمان باقی‌ماندهٔ اجرا رتبه می‌دهد و می‌تواند در هر لحظه قطع کند. این دو تنها برای ورودی‌های خاص (مثلاً حضور یک فرایند) بر هم منطبق می‌شوند. در حالت کلی برنامه و معیارهای متفاوتی تولید می‌کنند (جدول ۳ و مثال ۵)، بنابراین این مقاله هرگز یکی را مترادف دیگری نمی‌داند.

# فصل ۴: نحوهٔ عملکرد و مراحل اجرایی

این فصل تعریف را به رویه‌ای گام‌به‌گام و یک فلوچارت تبدیل می‌کند.

## ۴٫۱ رویهٔ گام‌به‌گام

۱٫ دریافت اطلاعات فرایندها: برای هر فرایند شناسه، زمان ورود و زمان اجرا.

۲٫ ثبت زمان ورود (Arrival Time) هر فرایند.

۳٫ ثبت زمان اجرا (Burst Time) هر فرایند.

۴٫ تشکیل صف آماده: در زمان جاری t هر فرایند ناتمام با AT <= t وارد صف می‌شود.

۵٫ مقایسهٔ زمان اجرای فرایندهای صف.

۶٫ انتخاب فرایند دارای بیشترین زمان اجرا (تساوی: AT کوچک‌تر، سپس اندیس کوچک‌تر).

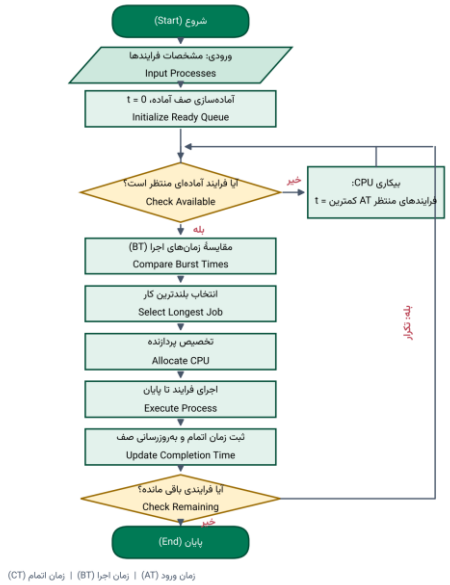
۷٫ تخصیص CPU به آن؛ زمان شروع ST = t.

۸/ مدیریت اتمام: CT = ST + BT و t = CT.

۹/ به‌روزرسانی صف: حذف فرایند تمام‌شده و پذیرش فرایندهای واردشده در حین اجرا.

۱۰/ تکرار مراحل ۴ تا ۹ تا پایان همهٔ فرایندها؛ اگر صف خالی بود t به ورود بعدی می‌رود.

## ۴٫۲ فلوچارت




**شکل ۲ـ** فلوچارت LJF غیرپیش‌دستانه همراه با شاخهٔ بیکاری CPU.

## ۴٫۳ شبه‌کد

**Algorithm 1: Non-preemptive LJF**

```
t = 0; done = 0
while done < n:
    ready = {p : p.AT <= t and not p.finished}
    if ready is empty:
        t = min(p.AT for p unfinished)    # CPU idle
        continue
    p* = argmax(ready, key = BT)          # tie: earlier AT, smaller index
    p*.ST = t;  t = t + p*.BT;  p*.CT = t
    p*.finished = true;  done = done + 1
```

### جمع‌بندی فصل

رویه میان تشکیل صف آماده، انتخاب بیشینهٔ BT و پیش‌بردن ساعت تناوب دارد؛ فاصله‌های بیکاری با پرش به ورود بعدی مدیریت می‌شوند.

# فصل ۵: مثال‌های عددی و حل گام‌به‌گام

پنج مثال موقعیت‌های اصلی را پوشش می‌دهند. در همهٔ آن‌ها ST زمان شروع و CT زمان اتمام هر فرایند است و TAT = CT - AT ، WT = TAT - BT و RT = ST - AT (صورت‌بندی در فصل ۶). همهٔ جدول‌ها را شبیه‌ساز مرجع تولید کرده و در ادامه دستی بازبینی شده‌اند.

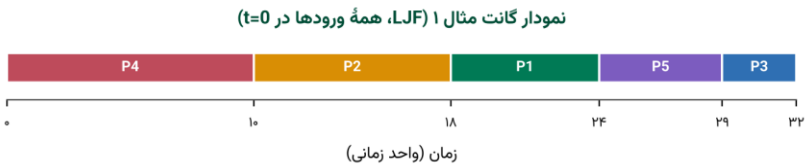
## ۵٫۱ مثال ۱: ورود هم‌زمان همهٔ فرایندها در زمان ۰

**جدول ۴.** مشخصات فرایندهای مثال ۱

| PID | AT | BT |
|---|---|---|
| P1 | ۰ | ۶ |
| P2 | ۰ | ۸ |
| P3 | ۰ | ۳ |
| P4 | ۰ | ۱۰ |
| P5 | ۰ | ۵ |

چون همهٔ فرایندها در t = ۰ حاضرند، LJF آن‌ها را به‌سادگی بر حسب BT نزولی مرتب می‌کند. ترتیب انتخاب: P4، P2، P1، P5، P3.

- در لحظهٔ t = ۰ صف آماده {P4 (۱۰), P2 (۸), P1 (۶), P5 (۵), P3 (۳)} است. بیشترین زمان اجرا (۱۰) متعلق به P4 است؛ بنابراین P4 در بازهٔ [۰, ۱۰) اجرا می‌شود.
- در لحظهٔ t = ۱۰ صف آماده {P2 (۸), P1 (۶), P5 (۵), P3 (۳)} است. بیشترین زمان اجرا (۸) متعلق به P2 است؛ بنابراین P2 در بازهٔ [۱۰, ۱۸) اجرا می‌شود.
- در لحظهٔ t = ۱۸ صف آماده {P1 (۶), P5 (۵), P3 (۳)} است. بیشترین زمان اجرا (۶) متعلق به P1 است؛ بنابراین P1 در بازهٔ [۱۸, ۲۴) اجرا می‌شود.
- در لحظهٔ t = ۲۴ صف آماده {P5 (۵), P3 (۳)} است. بیشترین زمان اجرا (۵) متعلق به P5 است؛ بنابراین P5 در بازهٔ [۲۴, ۲۹) اجرا می‌شود.
- در لحظهٔ t = ۲۹ صف آماده {P3 (۳)} است. بیشترین زمان اجرا (۳) متعلق به P3 است؛ بنابراین P3 در بازهٔ [۲۹, ۳۲) اجرا می‌شود.




**شکل ۳ـ** نمودار گانت مثال ۱.

**جدول ۵ـ** نتایج مثال ۱ تحت LJF

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۶ | ۱۸ | ۲۴ | ۲۴ | ۱۸ | ۱۸ |
| P2 | ۰ | ۸ | ۱۰ | ۱۸ | ۱۸ | ۱۰ | ۱۰ |
| P3 | ۰ | ۳ | ۲۹ | ۳۲ | ۳۲ | ۲۹ | ۲۹ |
| P4 | ۰ | ۱۰ | ۰ | ۱۰ | ۱۰ | ۰ | ۰ |
| P5 | ۰ | ۵ | ۲۴ | ۲۹ | ۲۹ | ۲۴ | ۲۴ |
| میانگین | | | | | ۲۲٫۶۰ | ۱۶٫۲۰ | ۱۶٫۲۰ |

تحلیل. P4 هیچ انتظاری ندارد اما P3 که کوتاه‌ترین کار است ۲۹ واحد منتظر می‌ماند، یعنی بیش از سه برابر زمان اجرای خودش. میانگین زمان انتظار ۱۶٫۲۰ = ۸۱/۵ و میانگین زمان برگشت = ۱۱۳/۵ ۲۲٫۶۰ است. بهره‌وری CPU برابر ۱۰۰٪ و گذردهی ۰٫۱۵۶ = ۵/۳۲ فرایند در واحد زمان است. چون ورودها هم‌زمان‌اند، برای هر فرایند WT با RT برابر است.

## ۵٫۲ مثال ۲: زمان‌های ورود متفاوت

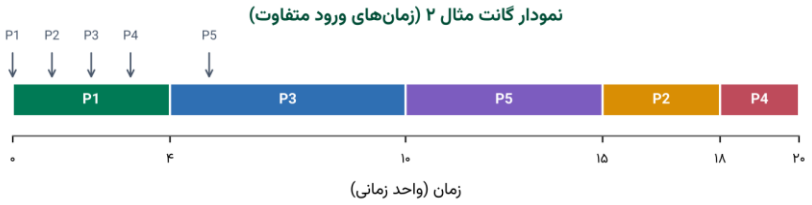
**جدول ۶ـ** مشخصات فرایندهای مثال ۲

| PID | AT | BT |
|---|---|---|
| P1 | ۰ | ۴ |
| P2 | ۱ | ۳ |
| P3 | ۲ | ۶ |
| P4 | ۳ | ۲ |
| P5 | ۵ | ۵ |

در اینجا انتخاب در هر اتمام به فرایندهای واردشده محدود است. در t = ۰ فقط P1 حاضر است و با وجود اینکه بلندترین فرایند کل نیست، شروع می‌شود.

- در لحظهٔ t = ۰ صف آماده {P1 (۴)} است. بیشترین زمان اجرا (۴) متعلق به P1 است؛ بنابراین P1 در بازهٔ [۰, ۴) اجرا می‌شود.

- در لحظهٔ t = ۴ صف آماده {P3 (۶), P2 (۳), P4 (۲)} است. بیشترین زمان اجرا (۶) متعلق به P3 است؛ بنابراین P3 در بازهٔ [۴, ۱۰) اجرا می‌شود.
- در لحظهٔ t = ۱۰ صف آماده {P5 (۵), P2 (۳), P4 (۲)} است. بیشترین زمان اجرا (۵) متعلق به P5 است؛ بنابراین P5 در بازهٔ [۱۰, ۱۵) اجرا می‌شود.
- در لحظهٔ t = ۱۵ صف آماده {P2 (۳), P4 (۲)} است. بیشترین زمان اجرا (۳) متعلق به P2 است؛ بنابراین P2 در بازهٔ [۱۵, ۱۸) اجرا می‌شود.
- در لحظهٔ t = ۱۸ صف آماده {P4 (۲)} است. بیشترین زمان اجرا (۲) متعلق به P4 است؛ بنابراین P4 در بازهٔ [۱۸, ۲۰) اجرا می‌شود.




**شکل ۴ـ** نمودار گانت مثال ۲ (پیکان‌ها ورودها را نشان می‌دهند).

**جدول ۷ـ** محاسبهٔ زمان اتمام در مثال ۲

| PID | ST | BT | BT + ST = CT |
|---|---|---|---|
| P1 | ۰ | ۴ | ۰ + ۴ = ۴ |
| P2 | ۱۵ | ۳ | ۱۵ + ۳ = ۱۸ |
| P3 | ۴ | ۶ | ۴ + ۶ = ۱۰ |
| P4 | ۱۸ | ۲ | ۱۸ + ۲ = ۲۰ |
| P5 | ۱۰ | ۵ | ۱۰ + ۵ = ۱۵ |

**جدول ۸ـ** محاسبهٔ زمان برگشت در مثال ۲

| PID | CT | AT | AT - CT = TAT |
|---|---|---|---|
| P1 | ۴ | ۰ | ۴ - ۰ = ۴ |
| P2 | ۱۸ | ۱ | ۱۸ - ۱ = ۱۷ |
| P3 | ۱۰ | ۲ | ۱۰ - ۲ = ۸ |
| P4 | ۲۰ | ۳ | ۲۰ - ۳ = ۱۷ |
| P5 | ۱۵ | ۵ | ۱۵ - ۵ = ۱۰ |

**جدول ۹ـ** محاسبهٔ زمان انتظار در مثال ۲

| PID | TAT | BT | BT - TAT = WT |
|---|---|---|---|
| P1 | ۴ | ۴ | ۴ - ۴ = ۰ |
| P2 | ۱۷ | ۳ | ۱۷ - ۳ = ۱۴ |
| P3 | ۸ | ۶ | ۸ - ۶ = ۲ |
| P4 | ۱۷ | ۲ | ۱۷ - ۲ = ۱۵ |
| P5 | ۱۰ | ۵ | ۱۰ - ۵ = ۵ |

**جدول ۱۰ـ** خلاصهٔ نتایج مثال ۲

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۴ | ۰ | ۴ | ۴ | ۰ | ۰ |
| P2 | ۱ | ۳ | ۱۵ | ۱۸ | ۱۷ | ۱۴ | ۱۴ |
| P3 | ۲ | ۶ | ۴ | ۱۰ | ۸ | ۲ | ۲ |
| P4 | ۳ | ۲ | ۱۸ | ۲۰ | ۱۷ | ۱۵ | ۱۵ |
| P5 | ۵ | ۵ | ۱۰ | ۱۵ | ۱۰ | ۵ | ۵ |
| میانگین | | | | | ۱۱٫۲۰ | ۷٫۲۰ | ۷٫۲۰ |

تحلیل. دو کار کوتاه‌تر، P4 (BT برابر ۲) و P2 (BT برابر ۳)، آخر تمام می‌شوند و به‌ترتیب ۱۵ و ۱۴ واحد منتظر می‌مانند؛ الگوی معمول .LJF میانگین زمان انتظار ۷٫۲۰ = ۳۶/۵ است. برای مقایسه، FCFS مقدار ۵٫۶۰ و SJF مقدار ۴٫۴۰ را روی همین بار کاری می‌دهد.

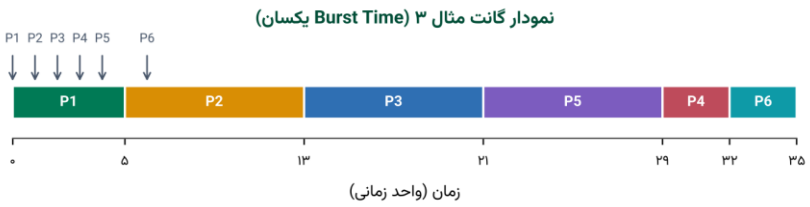
## ۵٫۳ مثال ۳: Burst Time یکسان

**جدول ۱۱ـ** مشخصات فرایندهای مثال ۳

| PID | AT | BT |
|---|---|---|
| P1 | ۰ | ۵ |
| P2 | ۱ | ۸ |
| P3 | ۲ | ۸ |
| P4 | ۳ | ۳ |
| P5 | ۴ | ۸ |
| P6 | ۶ | ۳ |

سه فرایند (P2، P3 و P5) بزرگ‌ترین BT یعنی ۸ را دارند و دو فرایند (P4 و P6) کوچک‌ترین یعنی ۳ را. با قاعدهٔ «ورود زودتر» تساوی‌ها به ترتیب ورود حل می‌شوند.

- در لحظهٔ t = ۰ صف آماده {P1 (۵)} است. بیشترین زمان اجرا (۵) متعلق به P1 است؛ بنابراین P1 در بازهٔ [۰, ۵) اجرا می‌شود.
- در لحظهٔ t = ۵ صف آماده {P2 (۸), P3 (۸), P5 (۸), P4 (۳)} است. بیشترین زمان اجرا (۸) متعلق به P2 است؛ بنابراین P2 در بازهٔ [۵, ۱۳) اجرا می‌شود.
- در لحظهٔ t = ۱۳ صف آماده {P3 (۸), P5 (۸), P4 (۳), P6 (۳)} است. بیشترین زمان اجرا (۸) متعلق به P3 است؛ بنابراین P3 در بازهٔ [۱۳, ۲۱) اجرا می‌شود.
- در لحظهٔ t = ۲۱ صف آماده {P5 (۸), P4 (۳), P6 (۳)} است. بیشترین زمان اجرا (۸) متعلق به P5 است؛ بنابراین P5 در بازهٔ [۲۱, ۲۹) اجرا می‌شود.
- در لحظهٔ t = ۲۹ صف آماده {P4 (۳), P6 (۳)} است. بیشترین زمان اجرا (۳) متعلق به P4 است؛ بنابراین P4 در بازهٔ [۲۹, ۳۲) اجرا می‌شود.
- در لحظهٔ t = ۳۲ صف آماده {P6 (۳)} است. بیشترین زمان اجرا (۳) متعلق به P6 است؛ بنابراین P6 در بازهٔ [۳۲, ۳۵) اجرا می‌شود.




**شکل ۵ـ** نمودار گانت مثال ۳ (BT یکسان).

**جدول ۱۲ـ** نتایج مثال ۳ تحت LJF

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۵ | ۰ | ۵ | ۵ | ۰ | ۰ |
| P2 | ۱ | ۸ | ۵ | ۱۳ | ۱۲ | ۴ | ۴ |
| P3 | ۲ | ۸ | ۱۳ | ۲۱ | ۱۹ | ۱۱ | ۱۱ |
| P4 | ۳ | ۳ | ۲۹ | ۳۲ | ۲۹ | ۲۶ | ۲۶ |
| P5 | ۴ | ۸ | ۲۱ | ۲۹ | ۲۵ | ۱۷ | ۱۷ |
| P6 | ۶ | ۳ | ۳۲ | ۳۵ | ۲۹ | ۲۶ | ۲۶ |
| میانگین | | | | | ۱۹٫۸۳ | ۱۴٫۰۰ | ۱۴٫۰۰ |

تحلیل. درون هر گروه مساوی برنامه همان FCFS است؛ پس P2، P3 و P5 به ترتیب ورود اجرا می‌شوند و P4 و P6 نیز. میانگین زمان انتظار ۱۴٫۰۰ = ۸۴/۶ است. قاعدهٔ دیگر، مثلاً کوچک‌ترین

شمارهٔ فرایند، در اینجا همان ترتیب را می‌دهد چون ترتیب ورود و اندیس منطبق‌اند؛ اما در حالت کلی قاعدهٔ شکستن تساوی می‌تواند زمان انتظار فرایندها را تغییر دهد و باید صریحاً بیان شود.

## ۵٫۴ مثال ۴: ده فرایند و تحلیل صف آماده

**جدول ۱۳ـ** مشخصات فرایندهای مثال ۴

| PID | AT | BT |
|---|---|---|
| P1 | ۰ | ۷ |
| P2 | ۱ | ۴ |
| P3 | ۲ | ۹ |
| P4 | ۳ | ۳ |
| P5 | ۴ | ۶ |
| P6 | ۶ | ۸ |
| P7 | ۸ | ۲ |
| P8 | ۹ | ۵ |
| P9 | ۱۲ | ۱۰ |
| P10 | ۱۴ | ۱ |

**جدول ۱۴ـ** صف آماده در هر لحظهٔ تصمیم در مثال ۴ (مرتب بر حسب BT نزولی)

| t | صف آماده (BT) | منتخب |
|---|---|---|
| ۰ | {P1 (۷)} | P1 |
| ۷ | {P3 (۹), P6 (۸), P5 (۶), P2 (۴), P4 (۳)} | P3 |
| ۱۶ | {P9 (۱۰), P6 (۸), P5 (۶), P8 (۵), P2 (۴), P4 (۳), P7 (۲), P10 (۱)} | P9 |
| ۲۶ | {P6 (۸), P5 (۶), P8 (۵), P2 (۴), P4 (۳), P7 (۲), P10 (۱)} | P6 |
| ۳۴ | {P5 (۶), P8 (۵), P2 (۴), P4 (۳), P7 (۲), P10 (۱)} | P5 |
| ۴۰ | {P8 (۵), P2 (۴), P4 (۳), P7 (۲), P10 (۱)} | P8 |
| ۴۵ | {P2 (۴), P4 (۳), P7 (۲), P10 (۱)} | P2 |
| ۴۹ | {P4 (۳), P7 (۲), P10 (۱)} | P4 |
| ۵۲ | {P7 (۲), P10 (۱)} | P7 |
| ۵۴ | {P10 (۱)} | P10 |




**شکل ۶ـ** نمودار گانت مثال ۴.

**جدول ۱۵ـ** نتایج مثال ۴ تحت LJF

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۷ | ۰ | ۷ | ۷ | ۰ | ۰ |
| P2 | ۱ | ۴ | ۴۵ | ۴۹ | ۴۸ | ۴۴ | ۴۴ |
| P3 | ۲ | ۹ | ۷ | ۱۶ | ۱۴ | ۵ | ۵ |
| P4 | ۳ | ۳ | ۴۹ | ۵۲ | ۴۹ | ۴۶ | ۴۶ |
| P5 | ۴ | ۶ | ۳۴ | ۴۰ | ۳۶ | ۳۰ | ۳۰ |
| P6 | ۶ | ۸ | ۲۶ | ۳۴ | ۲۸ | ۲۰ | ۲۰ |
| P7 | ۸ | ۲ | ۵۲ | ۵۴ | ۴۶ | ۴۴ | ۴۴ |
| P8 | ۹ | ۵ | ۴۰ | ۴۵ | ۳۶ | ۳۱ | ۳۱ |
| P9 | ۱۲ | ۱۰ | ۱۶ | ۲۶ | ۱۴ | ۴ | ۴ |
| P10 | ۱۴ | ۱ | ۵۴ | ۵۵ | ۴۱ | ۴۰ | ۴۰ |
| میانگین | | | | | ۳۱٫۹۰ | ۲۶٫۴۰ | ۲۶٫۴۰ |

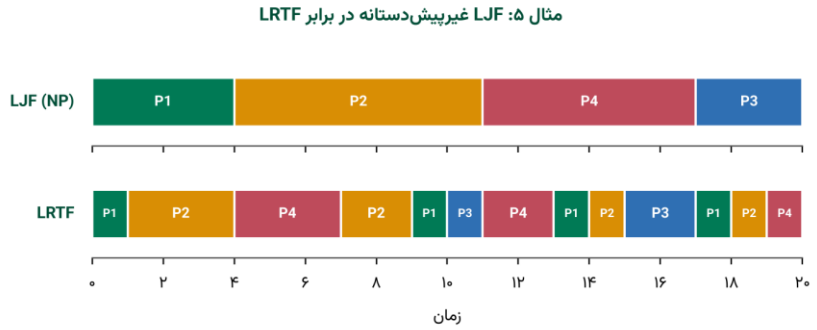
تحلیل. صف آماده از ۱ فرایند در t = ۰ به حداکثر ۸ فرایند در t = ۱۶ می‌رسد و سپس در هر تصمیم یکی کم می‌شود. سه کار کوتاه‌تر یعنی P10 (BT برابر ۱)، P7 (BT برابر ۲) و P4 (BT برابر ۳) آخر تمام می‌شوند و ۴۰، ۴۴ و ۴۶ واحد منتظر می‌مانند که به‌ترتیب ۴۰، ۲۲ و ۱۵ برابر زمان اجرای خودشان است. بر اساس قانون لیتل، میانگین تعداد فرایندهای منتظر برابر مجموع زمان انتظار تقسیم بر زمان سپری‌شده است: ۲۶۴/۵۵ = ۴٫۸۰. میانگین زمان انتظار ۲۶٫۴۰ و میانگین زمان برگشت ۳۱٫۹۰ است.

## ۵٫۵ مثال ۵: نسخهٔ پیش‌دستانه (LRTF) در برابر LJF غیرپیش‌دستانه

**جدول ۱۶ـ** مشخصات فرایندهای مثال ۵

| PID | AT | BT |
|---|---|---|
| P1 | ۰ | ۴ |
| P2 | ۱ | ۷ |
| P3 | ۲ | ۳ |
| P4 | ۴ | ۶ |

LJF غیرپیش‌دستانه P1 را ۴ واحد اجرا می‌کند، چون هنگام شروع فرایند بلندتری حاضر نیست. وقتی P2 (BT برابر ۷) در t = ۱ می‌رسد باید تا t = ۴ صبر کند. LRTF در هر واحد زمان‌های باقی‌مانده را مقایسه می‌کند: در t = ۱ زمان باقی‌ماندهٔ P2 برابر ۷ و P1 برابر ۳ است، پس P2 جلو می‌افتد و در بازهٔ [۱, ۴) اجرا می‌شود و زمان باقی‌ماندهٔ آن از ۷ به ۴ می‌رسد. وقتی P4 در t = ۴ با زمان باقی‌ماندهٔ ۶ می‌رسد و P2 باقی‌ماندهٔ ۴ دارد، P4 جلو می‌افتد و همین‌طور ادامه می‌یابد؛ هرگاه فرایند در حال اجرا در بیشینهٔ زمان باقی‌مانده مساوی باشد، همان ادامه می‌دهد تا تعویض بی‌مورد رخ ندهد.




**شکل ۷ـ** مثال ۵: LJF غیرپیش‌دستانه (بالا) در برابر LRTF (پایین).

**جدول ۱۷ـ** نتایج مثال ۵: LJF (غیرپیش‌دستانه)

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۴ | ۰ | ۴ | ۴ | ۰ | ۰ |
| P2 | ۱ | ۷ | ۴ | ۱۱ | ۱۰ | ۳ | ۳ |
| P3 | ۲ | ۳ | ۱۷ | ۲۰ | ۱۸ | ۱۵ | ۱۵ |
| P4 | ۴ | ۶ | ۱۱ | ۱۷ | ۱۳ | ۷ | ۷ |
| میانگین | | | | | ۱۱٫۲۵ | ۶٫۲۵ | ۶٫۲۵ |

**جدول ۱۸ـ** نتایج مثال ۵: LRTF (پیش‌دستانه)

| PID | AT | BT | ST | CT | TAT | WT | RT |
|---|---|---|---|---|---|---|---|
| P1 | ۰ | ۴ | ۰ | ۱۸ | ۱۸ | ۱۴ | ۰ |
| P2 | ۱ | ۷ | ۱ | ۱۹ | ۱۸ | ۱۱ | ۰ |
| P3 | ۲ | ۳ | ۱۰ | ۱۷ | ۱۵ | ۱۲ | ۸ |
| P4 | ۴ | ۶ | ۴ | ۲۰ | ۱۶ | ۱۰ | ۰ |
| میانگین | | | | | ۱۶٫۷۵ | ۱۱٫۷۵ | ۲٫۰۰ |

تحلیل. در LRTF هر چهار فرایند بین t = ۱۷ و t = ۲۰ تمام می‌شوند، در حالی که در LJF در ۴، ۱۱، ۱۷ و ۲۰ پایان می‌یابند؛ یعنی LRTF طبق نظریه زمان‌های پایان را همسان می‌کند، به بهای ۹ پیش‌دستی. میانگین زمان انتظار آن (۱۱٫۷۵) بسیار بیشتر از LJF (۶٫۲۵) است، ولی میانگین زمان پاسخ آن کمتر است (۲٫۰۰ در برابر ۶٫۲۵) زیرا جز P3 هر فرایند اندکی پس از ورود CPU می‌گیرد. در LRTF مقدار ST اولین شروع، CT آخرین لحظهٔ اتمام است و رابطهٔ BT - TAT = WT با وجود اجرای تکه‌تکه برقرار می‌ماند. زمان اتمام کل در هر دو حالت ۲۰ است، چون CPU بیکاری ندارد.

### جمع‌بندی فصل

پنج بار کاری، LJF را به‌صورت مرتب‌سازی نزولی (مثال ۱)، انتخاب محدود (مثال ۲)، انتخاب وابسته به شکستن تساوی (مثال ۳)، صفی که انباشته می‌شود و کارهای کوتاه را به گرسنگی می‌کشاند (مثال ۴) و تمایز روشن از LRTF (مثال ۵) نشان می‌دهند.

# فصل ۶: فرمول‌ها و تحلیل ریاضی

همهٔ معیارها برای مجموعه‌ای از n فرایند تعریف می‌شوند. AT_i، BT_i، ST_i و CT_i به‌ترتیب زمان ورود، اجرا، اولین شروع و اتمام فرایند i هستند.

## ۶٫۱ زمان برگشت (Turnaround Time)

$$\text{TAT} = \text{Completion Time} - \text{Arrival Time} \tag{۱}$$

زمان برگشت کل زمانی است که فرایند از ارسال تا اتمام در سیستم می‌گذراند و نگاه دسته‌ای به تأخیر است. مثال: در مثال ۲، P2 در ۱ می‌رسد و در ۱۸ تمام می‌شود، پس TAT = ۱۸ - ۱ = ۱۷. مقدار بسیار بزرگ‌تر از BT یعنی فرایند بیشتر عمرش را منتظر بوده است.

## ۶٫۲ زمان انتظار (Waiting Time)

$$WT = \text{Turnaround Time} - \text{Burst Time} \quad (۲)$$

زمان انتظار بخشی از زمان برگشت است که در صف آماده سپری می‌شود. در زمان‌بند غیرپیش‌دستانه برابر اولین شروع منهای ورود نیز هست. مثال: P4 در مثال ۲ دارای TAT = ۱۷ و BT = ۲ است، پس WT = ۱۵. این معیاری است که LJF در آن بدترین عملکرد را دارد.

## ۶٫۳ میانگین زمان انتظار

$$AWT = (\text{sum of } WT_i) / n \quad (۳)$$

AWT هزینهٔ انتظار کل سیستم را خلاصه می‌کند و معیاری است که SJF کمینه‌اش می‌کند. مثال: در مثال ۱، AWT = (۱۸ + ۱۰ + ۲۹ + ۰ + ۲۴)/۵ = ۸۱/۵ = ۱۶٫۲. مقایسه فقط برای بار کاری یکسان معنا دارد.

## ۶٫۴ میانگین زمان برگشت

$$ATAT = (\text{sum of } TAT_i) / n \quad (۴)$$

میانگین زمان برگشت، میانگین زمان رسیدن به اتمام است. مثال: مثال ۲ مقدار ATAT = (۴ + ۱۷ + ۸ + ۱۷ + ۱۰)/۵ = ۵۶/۵ = ۱۱٫۲ را می‌دهد. چون TAT_i = WT_i + BT_i ، همواره ATAT = AWT + mean(BT) برقرار است (۱۱٫۲ = ۷٫۲ + ۴٫۰).

## ۶٫۵ زمان پاسخ (Response Time)

$$RT = \text{First CPU Start Time} - \text{Arrival Time} \quad (۵)$$

زمان پاسخ نشان می‌دهد فرایند چقدر زود اولین CPU را می‌گیرد که برای کاربرد تعاملی مهم است. برای الگوریتم‌های غیرپیش‌دستانه RT = WT. مثال: P5 در مثال ۲ در ۵ می‌رسد و در ۱۰ شروع می‌کند، پس RT = ۵. در LRTF مقدار RT می‌تواند بسیار کمتر از WT باشد (مثال ۵).

## ۶٫۶ بهره‌وری CPU و گذردهی

$$\text{CPU Utilization} = (\text{CPU Busy Time} / \text{Total Elapsed Time}) \times 100 \quad (۶)$$

$$\text{Throughput} = \text{Number of Completed Processes} / \text{Total Elapsed Time} \tag{۷}$$

بهره‌وری نشان می‌دهد چه بخشی از بازه صرف اجرا می‌شود و گذردهی نرخ اتمام را می‌سنجد. مثال: P1 با (AT 0, BT 3) و P2 با (AT 10, BT 4). LJF، P1 را در [۰, ۳) و P2 را در [۱۰, ۱۴) اجرا می‌کند؛ CPU مدت ۷ واحد بیکار است، پس بهره‌وری برابر ۵۰ = ۱۰۰ × ۷/۱۴٪ و گذردهی ۰٫۱۴۳ = ۲/۱۴ در واحد زمان است. در مثال ۱ فاصله‌ای نیست، پس بهره‌وری ۱۰۰٪ و گذردهی ۰٫۱۵۶ = ۵/۳۲ است. وقتی بیکاری رخ نمی‌دهد، هر دو معیار از ترتیب اجرا مستقل‌اند و LJF را از SJF تفکیک نمی‌کنند.

**جدول ۱۹ـ** خلاصهٔ فرمول‌ها

| معیار | فرمول | هدف | مقدار در مثال ۲ |
|---|---|---|---|
| TAT | AT - CT | کمینه | P2: ۱۷ |
| WT | BT - TAT | کمینه | P4: ۱۵ |
| AWT | n / WTΣ | کمینه | ۷٫۲ |
| ATAT | n / TATΣ | کمینه | ۱۱٫۲ |
| RT | AT - ST | کمینه | P5: ۵ |
| Utilization | ۱۰۰ × Elapsed / Busy | بیشینه | ۱۰۰٪ |
| Throughput | Elapsed / Completed | بیشینه | ۰٫۲۵ |

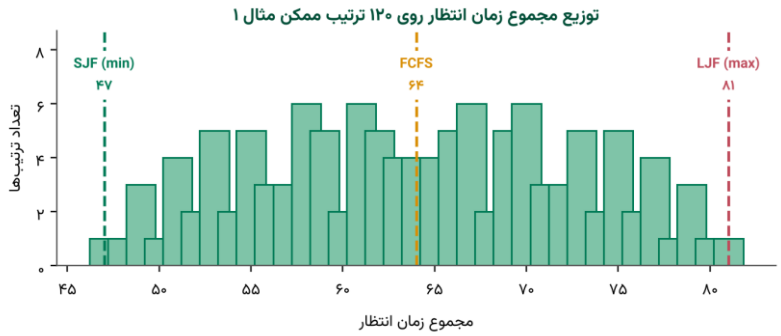
## ۶٫۷ تحلیل بهینگی: چرا LJF میانگین زمان انتظار را بیشینه می‌کند

n کار را در نظر بگیرید که همه در زمان ۰ آماده‌اند و به ترتیب σ(۱) تا σ(n) با زمان‌های اجرای b_۱, b_n, ... اجرا می‌شوند. کار در جایگاه k منتظر همهٔ کارهای قبلی است، پس هر b_k به تعداد کارهای بعدی شمرده می‌شود:

$$W(\sigma) = \Sigma_\{k=1..n\}\ (n - k) \cdot b_k \tag{۸}$$

دو کار مجاور x (جایگاه k) و y (جایگاه k + ۱) را جابه‌جا کنید. تغییر W برابر (x - y) است، زیرا جایگاه زودتر ضریب بزرگ‌تری دارد. اگر x > y باشد، قرار دادن x در جلو W را بزرگ‌تر می‌کند. تکرار چنین جابه‌جایی‌ها نشان می‌دهد ترتیب نزولی (LJF) W را بیشینه و ترتیب صعودی (SJF) آن را کمینه می‌کند؛ این حالت خاصی از نامساوی بازآرایی است [۵]. برای مثال ۱ جست‌وجوی کامل روی همهٔ ۵! = ۱۲۰ ترتیب این را تأیید می‌کند: مجموع زمان انتظار از ۴۷ (SJF) تا ۸۱ (LJF) تغییر می‌کند و FCFS

برابر ۶۴ است (شکل ۸). پس LJF هنگام ورود هم‌زمان، بدترین ترتیب غیرپیش‌دستانه از نظر میانگین زمان انتظار است.




**شکل ۸ـ** مجموع زمان انتظار روی هر ۱۲۰ ترتیب ممکن بار کاری مثال ۱.

## ۶٫۸ جایی که «بلندترین اول» خوب است: زمان اتمام کل روی ماشین‌های موازی

روی m پردازندهٔ مشابه هدف معمولاً زمان اتمام کل (Makespan) است، یعنی لحظهٔ پایان آخرین کار. تخصیص کارها به ترتیب نزولی به کم‌بارترین ماشین همان قاعدهٔ LPT است؛ گراهام ثابت کرد زمان اتمام کل آن حداکثر ((۴/۳ - ۱/(۳m)) برابر مقدار بهینه است [۷]. این تضمین علاقهٔ عملی به ترتیب بلندترین‌اول را توضیح می‌دهد و نشان می‌دهد رفتار ضعیف آن در انتظار روی یک CPU بهای بهینه‌سازی هدفی دیگر است.

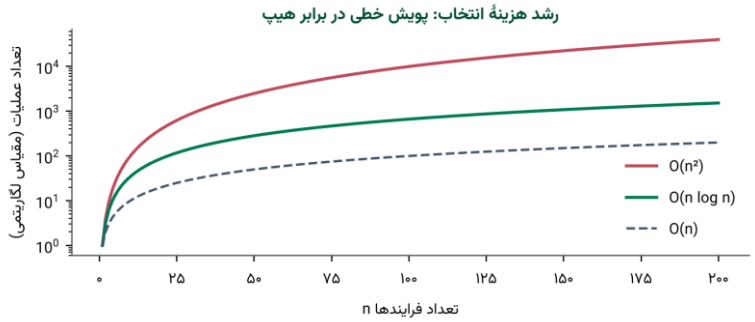
## ۶٫۹ پیچیدگی زمانی و فضایی

n را تعداد فرایندها بگیرید. پیاده‌سازی ساده در هر یک از n تصمیم کل فهرست را پویش می‌کند: O(n) برای هر تصمیم و در مجموع O(n²). فهرست مرتب بر حسب BT انتخاب را O(۱) می‌کند ولی درج فرایند واردشده O(n) هزینه دارد و در بدترین حالت مجموع باز O(n²) است. هیپ بیشینهٔ دودویی با کلید BT درج و استخراج را در O(log n) انجام می‌دهد؛ اگر فرایندها ابتدا بر حسب ورود در O(n log n) مرتب شوند، کل برنامه در O(n log n) محاسبه می‌شود [۴]. فضا در همهٔ حالت‌ها O(n) است. شکل ۹ رشد پویش خطی و انتخاب مبتنی بر هیپ را مقایسه می‌کند.

**جدول ۲۰ـ** پیچیدگی زمانی و فضایی پیاده‌سازی‌های LJF

| پیاده‌سازی | انتخاب | درج | زمان کل | فضا |
|---|---|---|---|---|
| فهرست نامرتب، پویش خطی | O(n) | O(۱) | O(n²) | O(n) |
| فهرست مرتب بر حسب BT | O(۱) | O(n) | O(n²) | O(n) |
| صف اولویت به‌صورت هیپ بیشینه | O(log n) | O(log n) | O(n log n) | O(n) |
| LRTF، شبیه‌سازی واحد زمانی | O(log n) در هر واحد | O(log n) | O(T log n)، T = مجموع BT | O(n) |

فرض‌ها: n فرایند، انتخاب مبتنی بر مقایسه، مرتب‌سازی یک‌بارهٔ ورودها در O(n log n). T مجموع زمان‌های اجراست، بنابراین کران LRTF شبه‌چندجمله‌ای است.




**شکل ۹ـ** رشد هزینهٔ کل انتخاب: پویش خطی (n²) در برابر هیپ (n log n).

### جمع‌بندی فصل

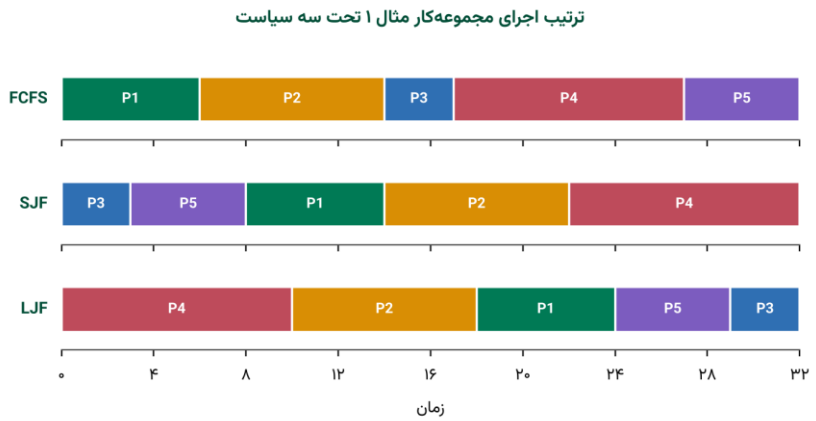
معیارها کمیت‌های خطی ساده‌اند؛ نتیجهٔ نظری کلیدی این است که برای ورود هم‌زمان LJF میانگین زمان انتظار را بیشینه و SJF کمینه می‌کند، در حالی که بلندترین‌اول برای زمان اتمام کل روی ماشین‌های موازی ابتکاری معتبر است.

# فصل ۷: نمودارها و تصاویر آموزشی

علاوه بر فلوچارت و نمودارهای گانت مثال‌ها، این فصل نمودارهای مقایسه‌ای را گرد می‌آورد. جدول ۲۱ همهٔ شکل‌های مقاله را با هدف هر یک فهرست می‌کند.

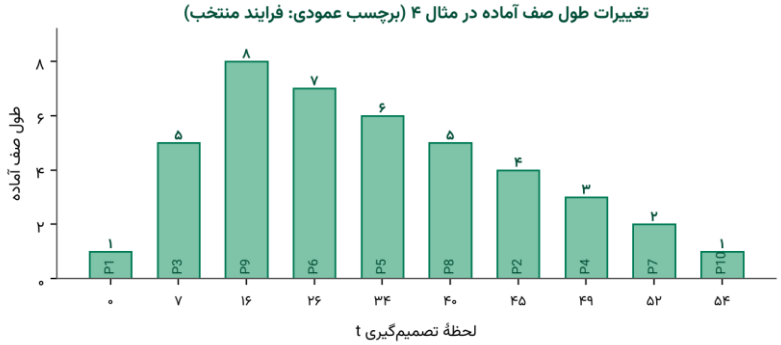
**جدول ۲۱ـ** فهرست شکل‌ها

| شکل | محتوا | هدف آموزشی |
|---|---|---|
| ۱ | نمودار حالات فرایند | مکان‌یابی تصمیم توزیع |
| ۲ | فلوچارت LJF | نمایش الگوریتم |
| ۳–۶ | نمودار گانت مثال‌های ۱ تا ۴ | دیداری‌سازی برنامه‌ها |
| ۷ | گانت LJF در برابر LRTF | نمایش اثر پیش‌دستی |
| ۸–۹ | هیستوگرام ترتیب‌ها، رشد پیچیدگی | پشتیبانی از تحلیل |
| ۱۰ | مقایسهٔ ترتیب اجرا | مقایسهٔ FCFS، SJF و LJF |
| ۱۱ | طول صف آماده (مثال ۴) | نمایش انباشت صف |
| ۱۲–۱۳ | انتظار / برگشت هر فرایند | مقایسهٔ سیاست‌ها برای هر کار |
| ۱۴–۱۵ | LJF با FCFS و LJF با SJF | کمّی‌سازی جریمه |



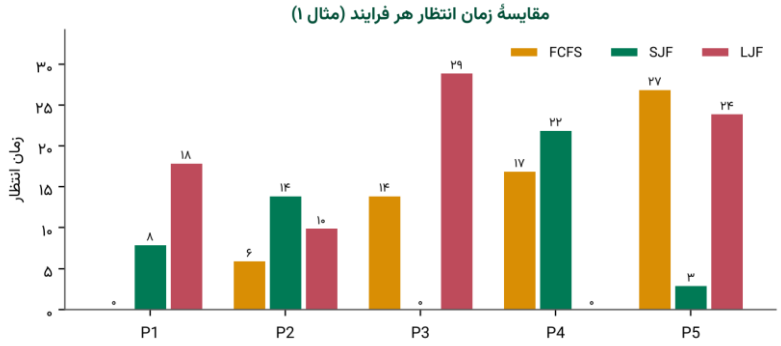

**شکل ۱۰ـ** ترتیب اجرای بار کاری مثال ۱ تحت FCFS، SJF و LJF.

شکل ۱۰ همان پنج فرایند را در سه ترتیب نشان می‌دهد. FCFS از شماره‌گذاری پیروی می‌کند، SJF کوتاه‌ترین را اول می‌برد و LJF دقیقاً وارونهٔ SJF است؛ زمان پایان آخرین فرایند، یعنی ۳۲، یکسان است و نشان می‌دهد ترتیب فقط تعیین می‌کند چه کسی منتظر می‌ماند، نه مقدار کار انجام‌شده.

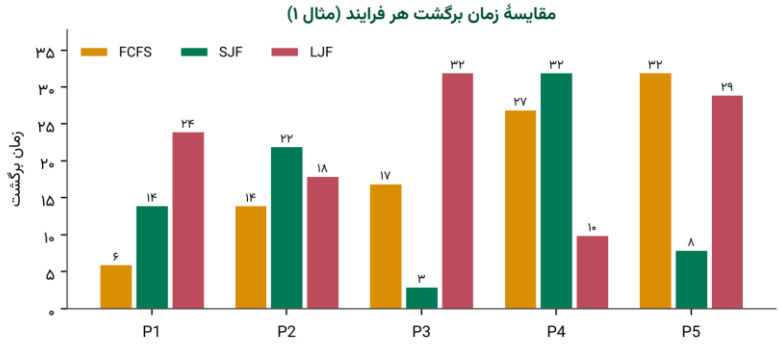



**شکل ۱۱ـ** طول صف آماده در لحظات تصمیم مثال ۴.

شکل ۱۱ نشان می‌دهد صف آماده در t = ۱۶ به اوج ۸ فرایند می‌رسد و سپس در هر تصمیم یکی کم می‌شود؛ دم بلند سمت راست را کارهای کوتاهی پر کرده‌اند که LJF به تعویق انداخته است.

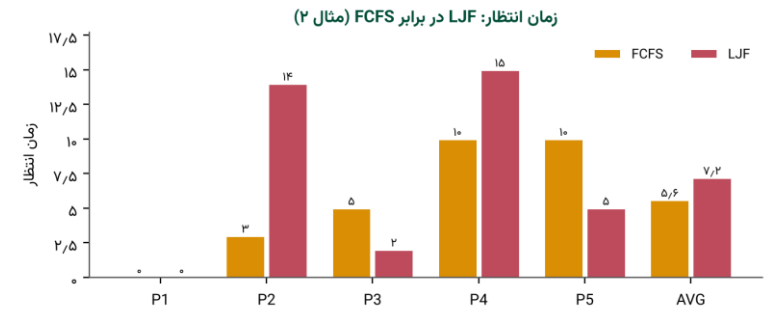



**شکل ۱۲ـ** زمان انتظار هر فرایند مثال ۱ تحت FCFS، SJF و LJF.

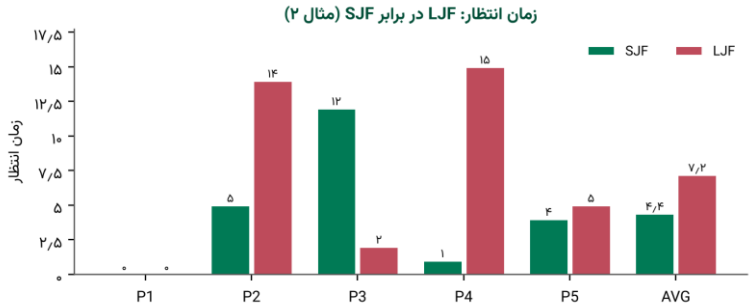



**شکل ۱۳ـ** زمان برگشت هر فرایند مثال ۱ تحت FCFS، SJF و LJF.

شکل‌های ۱۲ و ۱۳ نشان می‌دهند LJF کمترین انتظار را به بلندترین فرایند (P4: ۰) و بیشترین را به کوتاه‌ترین (P3: ۲۹) می‌دهد، عکس SJF. میانگین‌های زمان انتظار ۱۲٫۸۰ (FCFS)، ۹٫۴۰ (SJF) و ۱۶٫۲۰ (LJF) و میانگین‌های زمان برگشت متناظر ۱۹٫۲۰، ۱۵٫۸۰ و ۲۲٫۶۰ است.




**شکل ۱۴ـ** زمان انتظار مثال ۲ تحت LJF و FCFS.




**شکل ۱۵ـ** زمان انتظار مثال ۲ تحت LJF و SJF.

برای مثال ۲، میانگین زمان انتظار LJF برابر ۷٫۲۰ است در برابر ۵٫۶۰ برای FCFS و ۴٫۴۰ برای SJF؛ یعنی ۶۴٪ جریمه نسبت به SJF.

> **جمع‌بندی فصل**
>
> شکل‌ها تأیید می‌کنند که LJF کل کار را تغییر نمی‌دهد و فقط زمان انتظار را به سمت فرایندهای کوتاه بازتوزیع می‌کند.

# فصل ۸: مزایا و معایب الگوریتم LJF

## ۸٫۱ مزایا

- سادگی مفهوم. یک مقایسه (بیشترین BT) سیاست را تعریف می‌کند و توضیح و شبیه‌سازی آن آسان است.
- سهولت پیاده‌سازی در حالت پایه. یک پویش یا مرتب‌سازی کافی است (فصل ۱۰) و به تایمر یا پشتیبانی پیش‌دستی نیازی نیست.
- اولویت‌دهی به پردازش‌های طولانی. در مثال ۱ بزرگ‌ترین کار P4 در ۰ شروع می‌شود و با TAT برابر ۱۰ کوچک‌ترین زمان برگشت را دارد.
- امکان تحلیل رفتار. استدلال جابه‌جایی بخش ۶٫۷ نتایج دقیق می‌دهد و LJF را به نمونهٔ آزمون مناسبی برای اثبات‌ها و شبیه‌سازها تبدیل می‌کند.
- کاربرد آموزشی. مقایسهٔ LJF با SJF اثر کلید رتبه‌بندی را با ثابت‌نگه‌داشتن بقیهٔ عوامل جدا می‌کند.

## ۸٫۲ معایب

### ۸٫۲٫۱ افزایش انتظار پردازش‌های کوتاه

کوتاه‌ترین کارها منتظر همهٔ کارهای بلندتر می‌مانند. در مثال ۴ فرایند P10 (BT برابر ۱) مدت ۴۰ واحد منتظر می‌ماند؛ نسبت WT/BT برابر ۴۰ معیاری از ناخوشایندی شدید است.

### ۸٫۲٫۲ گرسنگی (Starvation) کارهای کوتاه

فرض کنید کاری بلند (BT برابر ۶) در حال اجراست که کاری کوتاه S (BT برابر ۲) در t = ۱ می‌رسد و پنج واحد زمانی بعد هر واحد یک کار بلند تازه وارد می‌شود. LJF همهٔ کارهای بلند را اول اجرا می‌کند و S در ۳۸ تمام می‌شود، پس از ۳۵ واحد انتظار؛ در حالی که SJF فقط ۵ واحد انتظار می‌دهد. اگر ورود کارهای بلند ادامه یابد، S بی‌نهایت منتظر می‌ماند: این همان گرسنگی است.

### ۸٫۲٫۳ میانگین زمان انتظار نامطلوب

همان‌طور که در بخش ۶٫۷ ثابت شد، هنگام ورود هم‌زمان LJF در میان همهٔ ترتیب‌های غیرپیش‌دستانه بیشترین AWT را می‌دهد؛ در مثال ۱ مقدار آن ۷۲٪ بیش از SJF است.

### ۸٫۲٫۴ وابستگی به تخمین Burst Time

این سیاست BT را پیش از اجرا لازم دارد. تخمین‌های حاصل از میانگین‌گیری نمایی می‌توانند اشتباه باشند و چون LJF هزینهٔ قرارگرفتن کار کوتاه پشت کار بلند را تشدید می‌کند، اثر خطاها از SJF بیشتر است.

### ۸٫۲٫۵ نامناسب بودن برای سیستم‌های تعاملی

کاربران تعاملی پاسخ اولیهٔ سریع انتظار دارند. چون LJF ممکن است هر درخواست کوتاه را پشت کارهای بلند دلخواه نگه دارد و غیرپیش‌دستانه است، زمان پاسخ کران ندارد. سیستم‌های اشتراک زمانی به‌جای آن از Round Robin یا صف‌های چندسطحی با بازخورد استفاده می‌کنند [۱, ۳].

### ۸٫۲٫۶ محدودیت‌های کاربردی

سیستم‌عامل‌های عمومی BT را از پیش نمی‌دانند، بار کاری ترکیبی تعاملی و دسته‌ای را سرویس می‌دهند و باید عدالت را تضمین کنند؛ از این رو LJF به‌ندرت سیاست اصلی آن‌هاست.

**جدول ۲۲ـ** مزایا و معایب LJF

| جنبه | مزیت | عیب |
|---|---|---|
| سادگی | سیاست تک‌قاعده‌ای، کد ساده | نیاز پنهان به دانستن BT |
| کارهای بلند | زود شروع می‌شوند، TAT کم | CPU را در دست می‌گیرند |
| کارهای کوتاه | – | انتظار طولانی، خطر گرسنگی |
| زمان انتظار | – | AWT بیشینه برای ورودهای رایج |
| تحلیل | نتایج دقیق و اثبات‌پذیر | نتایج نامطلوب‌اند |
| تعاملی بودن | – | زمان پاسخ بی‌کران |
| کاربرد عملی | آموزش، ابتکار برای Makespan موازی | در سیستم‌عامل‌های عمومی نادر |

### جمع‌بندی فصل

نقاط قوت LJF سادگی و تحلیل‌پذیری و نقاط ضعف آن (انتظار، گرسنگی، تخمین، تعاملی‌بودن) دلیل کاربرد عملی محدود آن است.

# فصل ۹: مقایسه با الگوریتم‌های دیگر

## ۹٫۱ مقایسهٔ کیفی

**جدول ۲۳.** مقایسهٔ جامع الگوریتم‌های زمان‌بندی

| الگوریتم | قاعدهٔ انتخاب | پیش‌دستی | مزایا | معایب | پیچیدگی | کاربرد |
|---|---|---|---|---|---|---|
| FCFS | زودترین ورود | خیر | ساده، ترتیب منصفانه | پدیدهٔ کاروانی | O(۱) برای هر تصمیم | دسته‌ای، پایه |
| SJF | کمترین BT | خیر | AWT کمینه (ورود مشترک) | گرسنگی، نیاز به BT | O(log n) با هیپ | دسته‌ای، نظری |
| SRTF | کمترین باقی‌مانده | بله | AWT بهینه با ورودها | تعویض زیاد، گرسنگی | O(log n) در هر رویداد | نظری، محک |
| Round Robin | چرخش، کوانتوم q | بله | پاسخ خوب، بدون گرسنگی | AWT بیشتر، سربار | O(۱) برای هر تصمیم | اشتراک زمانی |
| Priority | بالاترین اولویت | هر دو | سیاست انعطاف‌پذیر | گرسنگی بدون Aging | O(log n) با هیپ | بی‌درنگ، سیستم‌عامل عمومی |
| LJF | بیشترین BT | خیر | ساده، پشتیبان کارهای بلند | AWT بیشینه، گرسنگی | O(log n) با هیپ | آموزش، Makespan |
| LRTF | بیشترین باقی‌مانده | بله | زمان‌های پایان یکسان | تعویض زیاد، AWT بالا | O(T log n) | نظری، Makespan |

## ۹٫۲ رفتار در شرایط مختلف

- ورود مشترک: SJF برای AWT بهینه، LJF بدترین و FCFS میانه است.
- ورود متفاوت: SRTF بهینه است؛ پیش‌دستی به تازه‌واردهای کوتاه اجازهٔ اجرای فوری می‌دهد.
- طول‌های بسیار متغیر: FCFS و LJF بیشترین آسیب را می‌بینند، زیرا کارهای کوتاه پشت بلندها صف می‌کشند.
- بار تعاملی: Round Robin و اولویت همراه با Aging برترند؛ LJF و SJF نامناسب‌اند.
- Makespan روی چند CPU: بلندترین‌اول (LPT) از کوتاه‌ترین‌اول برتر است.

## ۹٫۳ مقایسهٔ عددی

**جدول ۲۴ـ** میانگین معیارهای همهٔ الگوریتم‌ها روی بار کاری مثال ۲

| الگوریتم | AWT | ATAT | ART | تعداد پیش‌دستی |
|---|---|---|---|---|
| FCFS | ۵٫۶۰ | ۹٫۶۰ | ۵٫۶۰ | ۰ |
| SJF | ۴٫۴۰ | ۸٫۴۰ | ۴٫۴۰ | ۰ |
| SRTF | ۴٫۴۰ | ۸٫۴۰ | ۴٫۴۰ | ۰ |
| RR (q = ۳) | ۷٫۲۰ | ۱۱٫۲۰ | ۳٫۸۰ | ۳ |
| Priority (NP) | ۴٫۸۰ | ۸٫۸۰ | ۴٫۸۰ | ۰ |
| LJF | ۷٫۲۰ | ۱۱٫۲۰ | ۷٫۲۰ | ۰ |
| LRTF | ۱۱٫۸۰ | ۱۵٫۸۰ | ۳٫۴۰ | ۸ |

اولویت‌ها برای Priority (عدد کمتر = اولویت بیشتر): P1 = ۳، P2 = ۱، P3 = ۴، P4 = ۲، P5 = ۵. تعداد پیش‌دستی یعنی چند بار فرایندی پیش از اتمام از CPU برداشته شده است؛ الگوریتم‌های غیرپیش‌دستانه صفر دارند.

روی این بار کاری SJF و SRTF بهترین AWT را دارند (۴٫۴۰)، FCFS (۵٫۶۰) و Priority (۴٫۸۰) نزدیک‌اند و LJF (۷٫۲۰، برابر Round Robin) از همهٔ آن‌ها بدتر است، جز LRTF (۱۱٫۸۰) که البته بهترین زمان پاسخ (۳٫۴۰) را دارد. زمان پاسخ Round Robin (۳٫۸۰) نیز، مطابق هدف طراحی‌اش، پایین است.

### جمع‌بندی فصل

LJF انتظار را با خدمت‌دهی زودهنگام به کارهای بلند معامله می‌کند؛ الگوریتم مناسب به این بستگی دارد که هدف میانگین انتظار، پاسخ، عدالت یا Makespan باشد.

# فصل ۱۰: پیاده‌سازی برنامه‌نویسی

برنامه‌های زیر LJF غیرپیش‌دستانه را پیاده‌سازی می‌کنند. آن‌ها تعداد فرایندها و سپس خطوطی به شکل `PID AT BT` را از ورودی استاندارد می‌خوانند و برای هر فرایند به ترتیب اجرا یک سطر، سپس نمودار گانت، معیارهای میانگین، بهره‌وری CPU و گذردهی را چاپ می‌کنند. هر سه روی ورودی‌های یکسان کامپایل و اجرا شدند و خروجی یکسان دادند. قاعدهٔ شکستن تساوی همان بخش ۲٫۴ است. کدها و توضیح‌های درون‌کد به انگلیسی نوشته شده‌اند تا مستقیماً قابل اجرا و قابل انتقال باشند.

## Python ۱۰٫۱

**Listing 1: ljf.py (Python 3.8+)**

```python
import sys
from dataclasses import dataclass


@dataclass
class Process:
    pid: str
    at: int  # arrival time
    bt: int  # burst time


def ljf(procs):
    """Non-preemptive Longest Job First. Returns (rows, gantt)."""
    pending = list(enumerate(procs))            # (input index, process)
    t, rows, gantt = 0, [], []
    while pending:
        ready = [(i, p) for i, p in pending if p.at <= t]
        if not ready:                           # CPU idle: jump to next arrival
            nxt = min(p.at for _, p in pending)
            gantt.append(("IDLE", t, nxt))
            t = nxt
            continue
        # longest BT first; ties -> earlier arrival -> smaller input index
        i, p = min(ready, key=lambda x: (-x[1].bt, x[1].at, x[0]))
        pending.remove((i, p))
        st, ct = t, t + p.bt
        rows.append(dict(pid=p.pid, at=p.at, bt=p.bt, st=st, ct=ct,
                         tat=ct - p.at, wt=ct - p.at - p.bt, rt=st - p.at))
        gantt.append((p.pid, st, ct))
        t = ct
    return rows, gantt


def lrtf(procs):
    """Preemptive variant (LRTF), unit-time simulation. Returns gantt."""
    rem = {p.pid: p.bt for p in procs}
    t, cur, gantt = 0, None, []
    while any(rem.values()):
        ready = [p for p in procs if p.at <= t and rem[p.pid] > 0]
        if not ready:
            t, cur = t + 1, None
            continue
        top = max(rem[p.pid] for p in ready)
        cands = [p for p in ready if rem[p.pid] == top]
        nxt = next((p for p in cands if p.pid == cur),
                   min(cands, key=lambda p: p.at))   # tie: keep running one
        if gantt and gantt[-1][0] == nxt.pid and gantt[-1][2] == t:
            gantt[-1] = (nxt.pid, gantt[-1][1], t + 1)
        else:
            gantt.append((nxt.pid, t, t + 1))
        rem[nxt.pid] -= 1
        cur, t = nxt.pid, t + 1
    return gantt


def main():
    tok = sys.stdin.read().split()
    n = int(tok[0]) if tok else 0
    if n == 0:
        print("No processes.")
        return
    procs = [Process(tok[1 + 3 * i], int(tok[2 + 3 * i]), int(tok[3 + 3 * i]))
             for i in range(n)]
    if any(p.at < 0 or p.bt <= 0 for p in procs):
        print("Invalid input: AT must be >= 0 and BT must be > 0")
        sys.exit(1)
    rows, gantt = ljf(procs)
    print(f"{'PID':<5}{'AT':>4}{'BT':>4}{'ST':>4}{'CT':>4}"
          f"{'TAT':>5}{'WT':>4}{'RT':>4}")
    for r in rows:
        print(f"{r['pid']:<5}{r['at']:>4}{r['bt']:>4}{r['st']:>4}{r['ct']:>4}"
              f"{r['tat']:>5}{r['wt']:>4}{r['rt']:>4}")
    print("Gantt: |" + "|".join(f"{g} {a}-{b}" for g, a, b in gantt) + "|")
    end = max(r["ct"] for r in rows)
    busy = sum(r["bt"] for r in rows)
    print(f"AWT={sum(r['wt'] for r in rows) / n:.2f}  "
          f"ATAT={sum(r['tat'] for r in rows) / n:.2f}  "
          f"ART={sum(r['rt'] for r in rows) / n:.2f}")
    print(f"CPU utilization={100.0 * busy / end:.2f}%  Throughput={n / end:.3f}")


if __name__ == "__main__":
    main()
```

**جدول ۲۵ـ** توضیح خط‌به‌خط برنامهٔ Python

| خطوط | توضیح |
|---|---|
| ۱–۹ | واردکردن کتابخانه‌ها و کلاس Process با فیلدهای pid، at (ورود) و bt (اجرا). |
| ۱۲–۱۵ | تابع ljf فهرست را می‌گیرد؛ pending زوج‌های (اندیس ورودی، فرایند) را نگه می‌دارد و t ساعت است. |
| ۱۶–۲۲ | صف آماده: همهٔ فرایندهای معلق با t. => at اگر خالی باشد CPU بیکار است: قطعهٔ IDLE ثبت و ساعت به ورود بعدی پرش می‌کند (مدیریت Arrival Time). |
| ۲۳–۲۵ | انتخاب: کمینهٔ کلید (-bt, at, index) یعنی بلندترین BT با اولویت ورود زودتر و اندیس کوچک‌تر؛ فرایند منتخب حذف می‌شود. |
| ۲۶–۳۱ | t = ST، CT = ST + BT، TAT = CT - AT، WT = TAT - BT و RT = ST - AT؛ ساعت به CT می‌رود. |
| ۳۴–۵۳ | تابع lrtf: شبیه‌سازی واحد زمانی جداگانه برای نسخهٔ پیش‌دستانه (نه LJF)؛ در تساوی فرایند در حال اجرا حفظ می‌شود. |
| ۵۶–۶۶ | خواندن ورودی؛ n = ۰ پیام می‌دهد؛ BT غیرمثبت یا AT منفی رد می‌شود (حالت‌های مرزی). |
| ۶۷–۷۹ | چاپ جدول، نمودار گانت، میانگین‌ها، بهره‌وری CPU (busy/end × ۱۰۰) و گذردهی (n/end). |

## C++ ۱۰٫۲

**Listing 2: ljf.cpp (C++17)**

```cpp
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Proc {
    string pid;
    int at, bt;
    int st = 0, ct = 0;
    bool done = false;
};
struct Seg { string pid; int a, b; };

int main() {
    int n;
    if (!(cin >> n) || n == 0) { cout << "No processes.\n"; return 0; }
    vector<Proc> p(n);
    for (auto &x : p) {
        cin >> x.pid >> x.at >> x.bt;
        if (x.at < 0 || x.bt <= 0) {
            cout << "Invalid input: AT must be >= 0 and BT must be > 0\n";
            return 1;
        }
    }
    vector<Seg> gantt;
    vector<int> order;
    int t = 0, finished = 0;
    while (finished < n) {
        int best = -1, nextArrival = 1 << 30;
        for (int i = 0; i < n; ++i) {
            if (p[i].done) continue;
            nextArrival = min(nextArrival, p[i].at);
            if (p[i].at > t) continue;              // not in the ready queue
            // longest BT; ties -> earlier arrival -> smaller input index
            if (best < 0 || p[i].bt > p[best].bt ||
                (p[i].bt == p[best].bt && p[i].at < p[best].at))
                best = i;
        }
        if (best < 0) {                             // CPU idle until next arrival
            gantt.push_back({"IDLE", t, nextArrival});
            t = nextArrival;
            continue;
        }
        p[best].st = t;
        t += p[best].bt;
        p[best].ct = t;
```

```
49          p[best].done = true;
50          ++finished;
51          order.push_back(best);
52          gantt.push_back({p[best].pid, p[best].st, p[best].ct});
53      }
54      printf("%-5s%4s%4s%4s%4s%5s%4s%4s\n", "PID", "AT", "BT", "ST", "CT",
55             "TAT", "WT", "RT");
56      double sw = 0, st = 0, sr = 0, busy = 0;
57      for (int i : order) {
58          const Proc &x = p[i];
59          int tat = x.ct - x.at, wt = tat - x.bt, rt = x.st - x.at;
60          printf("%-5s%4d%4d%4d%4d%5d%4d%4d\n", x.pid.c_str(), x.at, x.bt, x.st,
61                 x.ct, tat, wt, rt);
62          sw += wt; st += tat; sr += rt; busy += x.bt;
63      }
64      printf("Gantt: |");
65      for (auto &g : gantt) printf("%s %d-%d|", g.pid.c_str(), g.a, g.b);
66      printf("\nAWT=%.2f  ATAT=%.2f  ART=%.2f\n", sw / n, st / n, sr / n);
67      printf("CPU utilization=%.2f%%  Throughput=%.3f\n",
68             100.0 * busy / t, n / (double)t);
69      return 0;
70  }
```

**جدول ۲۶ـ** توضیح خط‌به‌خط برنامهٔ C++

| خطوط | توضیح |
|---|---|
| ۱–۱۴ | سرفایل‌ها و دو ساختار: Proc (داده و نتایج) و Seg (قطعهٔ گانت). |
| ۱۶–۲۶ | ورودی: n و رکوردهای فرایندها؛ مقادیر نامعتبر رد می‌شوند و n = ۰ پیام می‌دهد. |
| ۲۷–۴۰ | حلقهٔ اصلی. برای هر فرایند ناتمام زودترین ورود معلق دنبال می‌شود؛ فرایندهای با t < at کنار گذاشته می‌شوند (در صف آماده نیستند)؛ بهترین نامزد بیشترین bt را دارد و در تساوی ورود زودتر برنده است؛ مقایسهٔ اکید اندیس کوچک‌تر را نگه می‌دارد. |
| ۴۱–۴۵ | شاخهٔ بیکاری: اگر چیزی آماده نباشد IDLE ثبت و t به ورود بعدی می‌رود. |
| ۴۶–۵۳ | توزیع: t = ST؛ t =+ BT؛ t = CT؛ علامت‌گذاری پایان؛ ثبت ترتیب و قطعهٔ گانت. |
| ۵۴–۷۰ | خروجی: TAT = AT - CT، WT = BT - TAT، RT = AT - ST و سپس میانگین‌ها، بهره‌وری و گذردهی. |

## Java ۱۰٫۳

**Listing 3: LJF.java (Java 11+, run with: java LJF.java)**

```
1   import java.util.*;
2
3   public class LJF {
4       static class Proc {
5           String pid; int at, bt, st, ct; boolean done;
6           Proc(String pid, int at, int bt) {
7               this.pid = pid; this.at = at; this.bt = bt;
8           }
9       }
10
11      public static void main(String[] args) {
12          Scanner sc = new Scanner(System.in);
13          int n = sc.hasNextInt() ? sc.nextInt() : 0;
14          if (n == 0) { System.out.println("No processes."); return; }
15          Proc[] p = new Proc[n];
16          for (int i = 0; i < n; i++) {
17              p[i] = new Proc(sc.next(), sc.nextInt(), sc.nextInt());
18              if (p[i].at < 0 || p[i].bt <= 0) {
19                  System.out.println(
20                      "Invalid input: AT must be >= 0 and BT must be > 0");
21                  System.exit(1);
22              }
23          }
24          List<String> gantt = new ArrayList<>();
25          List<Integer> order = new ArrayList<>();
26          int t = 0, finished = 0;
27          while (finished < n) {
28              int best = -1, nextArrival = Integer.MAX_VALUE;
29              for (int i = 0; i < n; i++) {
30                  if (p[i].done) continue;
31                  nextArrival = Math.min(nextArrival, p[i].at);
32                  if (p[i].at > t) continue;          // not in the ready queue
```

```
                // longest BT; ties -> earlier arrival -> smaller input index
                if (best < 0 || p[i].bt > p[best].bt
                        || (p[i].bt == p[best].bt && p[i].at < p[best].at))
                    best = i;
            }
            if (best < 0) {                         // CPU idle until next arrival
                gantt.add("IDLE " + t + "-" + nextArrival);
                t = nextArrival;
                continue;
            }
            p[best].st = t;
            t += p[best].bt;
            p[best].ct = t;
            p[best].done = true;
            finished++;
            order.add(best);
            gantt.add(p[best].pid + " " + p[best].st + "-" + p[best].ct);
        }
        System.out.println(String.format("%-5s%4s%4s%4s%4s%5s%4s%4s",
                "PID", "AT", "BT", "ST", "CT", "TAT", "WT", "RT"));
        double sw = 0, stt = 0, sr = 0, busy = 0;
        for (int i : order) {
            Proc x = p[i];
            int tat = x.ct - x.at, wt = tat - x.bt, rt = x.st - x.at;
            System.out.println(String.format("%-5s%4d%4d%4d%4d%5d%4d%4d",
                    x.pid, x.at, x.bt, x.st, x.ct, tat, wt, rt));
            sw += wt; stt += tat; sr += rt; busy += x.bt;
        }
        System.out.println("Gantt: |" + String.join("|", gantt) + "|");
        System.out.println(String.format("AWT=%.2f  ATAT=%.2f  ART=%.2f",
                sw / n, stt / n, sr / n));
        System.out.println(String.format("CPU utilization=%.2f%%  Throughput=%.3f",
                100.0 * busy / t, n / (double) t));
    }
}
```

**جدول ۲۷.** توضیح خط‌به‌خط برنامهٔ Java

| خطوط | توضیح |
|---|---|
| ۹–۱ | کلاس Process با فیلدهای شناسه، AT، BT و ST و CT محاسبه‌شده. |
| ۲۳–۱۱ | خواندن n و فرایندها با Scanner؛ مقادیر نامعتبر یا n = ۰ پیش از زمان‌بندی مدیریت می‌شوند. |
| ۳۷–۲۴ | همان حلقهٔ انتخاب C++: فرایندهای نرسیده را رد می‌کند، بزرگ‌ترین BT را برمی‌گزیند و تساوی را با ورود زودتر و اندیس کوچک‌تر می‌شکند. |
| ۴۲–۳۸ | شاخهٔ بیکاری: افزودن ورودی IDLE و پیش‌بردن ساعت. |
| ۵۰–۴۳ | توزیع و اتمام: محاسبهٔ ST و CT و به‌روزرسانی فهرست ترتیب و فهرست گانت. |
| ۶۶–۵۱ | خروجی قالب‌بندی‌شدهٔ سطرها، نمودار گانت و معیارهای خلاصه. |

## ۱۰٫۴ ورودی و خروجی نمونه

**Sample input (Example 2)**

```
5
P1 0 4
P2 1 3
P3 2 6
P4 3 2
P5 5 5
```

**Sample output (identical in Python, C++ and Java)**

```
PID    AT  BT  ST  CT  TAT  WT  RT
P1      0   4   0   4    4   0   0
P3      2   6   4  10    8   2   2
P5      5   5  10  15   10   5   5
P2      1   3  15  18   17  14  14
P4      3   2  18  20   17  15  15
Gantt: |P1 0-4|P3 4-10|P5 10-15|P2 15-18|P4 18-20|
AWT=7.20  ATAT=11.20  ART=7.20
CPU utilization=100.00%  Throughput=0.250
```

خروجی جدول ۱۰ و شکل ۴ را بازتولید می‌کند: ترتیب P1، P3، P5، P2، P4، زمان‌های انتظار ۰، ۲، ۵، ۱۴ و ۱۵ و میانگین زمان انتظار ۷٫۲۰.

## ۱۰٫۵ حالت‌های مرزی و بررسی صحت نتایج

**Input with an idle gap (P1: AT 0, BT 3; P2: AT 10, BT 4) and its output**

```
2
P1 0 3
P2 10 4


PID    AT  BT  ST  CT  TAT  WT  RT
P1      0   3   0   3    3   0   0
P2     10   4  10  14    4   0   0
Gantt: |P1 0-3|IDLE 3-10|P2 10-14|
AWT=0.00  ATAT=3.50  ART=0.00
CPU utilization=50.00%  Throughput=0.143
```

- ورودی تهی (n = ۰): برنامه پیام «No processes.» را چاپ می‌کند و پایان می‌یابد.
- دادهٔ نامعتبر (BT ≤ ۰ یا AT < ۰): با پیام و کد خروج ۱ رد می‌شود.
- بیکاری CPU: ساعت به ورود بعدی می‌پرد و نمودار گانت قطعهٔ IDLE نشان می‌دهد (بهره‌وری ۵۰٫۰۰٪ در بالا).
- اولین ورود پس از ۰: یک فرایند با ورود ۷ قطعهٔ IDLE 0–۷ و انتظار صفر تولید می‌کند.
- تساوی‌ها: با ورود زودتر و سپس ترتیب ورودی حل می‌شوند، مانند مثال ۳.
- گردکردن: همهٔ مقادیر با ارقام اعشار ثابت چاپ می‌شوند؛ تساوی‌های دقیق دودویی در آخرین رقم ممکن است بین زبان‌ها متفاوت گرد شوند و به همین دلیل گذردهی با سه رقم اعشار چاپ می‌شود.

**جدول ۲۸ـ** راستی‌آزمایی پیاده‌سازی‌ها روی بارهای کاری مقاله

| ورودی | AWT | ATAT | بهره‌وری | یکسان در ۳ زبان | هم‌خوان با شبیه‌ساز |
|---|---|---|---|---|---|
| مثال ۱ | ۱۶٫۲۰ | ۲۲٫۶۰ | ۱۰۰٫۰۰٪ | بله | بله |
| مثال ۲ | ۷٫۲۰ | ۱۱٫۲۰ | ۱۰۰٫۰۰٪ | بله | بله |
| مثال ۳ | ۱۴٫۰۰ | ۱۹٫۸۳ | ۱۰۰٫۰۰٪ | بله | بله |
| مثال ۴ | ۲۶٫۴۰ | ۳۱٫۹۰ | ۱۰۰٫۰۰٪ | بله | بله |
| فاصلهٔ بیکاری | ۰٫۰۰ | ۳٫۵۰ | ۵۰٫۰۰٪ | بله | بله |
| یک فرایند، AT = ۷ | ۰٫۰۰ | ۴٫۰۰ | ۳۶٫۳۶٪ | بله | بله |

پیاده‌سازی Python علاوه بر آن قطعه‌به‌قطعه با شبیه‌ساز مرجع برای مثال‌های ۱ تا ۴ و بار کاری بیکاری مقایسه شد و تابع LRTF آن با برنامهٔ مثال ۵ سنجیده شد.

**جمع‌بندی فصل**

سه برنامهٔ مستقل روی همهٔ آزمون‌ها، از جمله حالت‌های مرزی، با هم و با مثال‌های حل‌شدهٔ دستی هم‌خوان‌اند.

# فصل ۱۱: مثال‌های کاربردی و مطالعات موردی

این فصل میان کاربرد نظری، که سیاست مدل یا ابتکار معتبر است، و کاربرد عملی در سیستم‌عامل‌های واقعی، که نادر است، تمایز می‌گذارد. ادعاها به منابع ذکرشده متکی‌اند.

## ۱۱٫۱ کاربرد نظری در برابر استفادهٔ عملی در سیستم‌عامل‌های واقعی

کتاب‌های درسی معتبر FCFS، SJF/SRTF، اولویت و Round Robin را همراه با صف‌های چندسطحی به‌عنوان مجموعهٔ پایه معرفی می‌کنند و یادآور می‌شوند SJF در سطح کوتاه‌مدت دقیقاً قابل پیاده‌سازی نیست، چون BT بعدی نامعلوم است [۱, ۲, ۳]. قواعد بلندترین‌اول جزو این مجموعهٔ استاندارد نیستند. زمان‌بندهای عمومی تولیدی به‌جای طول کار از سازوکارهای اولویت و عدالت استفاده می‌کنند؛ بنابراین LJF را باید مدل و ابتکار دانست، نه سیاستی که هسته‌های واقعی به‌عنوان الگوریتم اصلی برگزینند.

## ۱۱٫۲ پردازش دسته‌ای و سیستم‌های آفلاین

در پردازش دسته‌ای و آفلاین همهٔ کارها از پیش معلوم‌اند، تعاملی در کار نیست و هدف اغلب تمام‌کردن مجموعه‌ای از کارها هرچه زودتر روی چند کارگر است. در اینجا بلندترین‌اول همان قاعدهٔ LPT و ابتکار ساده و استاندارد برای کمینه‌سازی Makespan با تضمین $(4/3 - 1/(3m))$ است [۷, ۸]. برای یک کارگر با هدف کمینه‌سازی میانگین زمان اتمام، کوتاه‌ترین‌اول بهینه است [۵]؛ پس LJF انتخاب نادرستی است.

**جدول ۲۹ـ** مناسب بودن LJF در محیط‌های مختلف

| محیط | هدف اصلی | تناسب LJF | توضیح |
|---|---|---|---|
| دسته‌ای (یک CPU) | میانگین انتظار | ضعیف | SJF بهینه است [۵] |
| دسته‌ای (ماشین‌های موازی) | Makespan | ابتکار خوب | تضمین LPT [۷] |
| پردازش آفلاین | گذردهی کل مجموعه | متوسط | ترتیب فقط روی انتظار اثر دارد |
| شبیه‌سازهای آموزشی | توضیح مفهوم | بسیار خوب | تقابل با SJF |
| محاسبات طولانی | پایان متوازن | خوب در حالت موازی | نیازمند تخمین قابل‌اعتماد |
| سیستم‌های تعاملی | زمان پاسخ | نامناسب | تأخیر بی‌کران |

## ۱۱٫۳ محیط‌های شبیه‌سازی آموزشی

شبیه‌سازهای زمان‌بندی فرایند و تمرین‌های درسی معمولاً LJF را دارند، زیرا در چند خط پیاده‌سازی می‌شود و نتیجه‌ای خلاف شهود می‌دهد: الگوریتمی که بزرگ‌ترین کار را اول سرویس می‌دهد بدترین میانگین زمان انتظار را دارد. این موضوع آن را مثال مؤثری برای بحث دربارهٔ اهداف، بهینگی و گرسنگی می‌کند.

## ۱۱٫۴ محیط‌های محاسباتی با وظایف طولانی

وقتی وظایف ساعت‌ها اجرا می‌شوند و پیش‌دستی گران است (مثلاً شبیه‌سازی‌ها یا آموزش مدل‌های بزرگ)، قاعدهٔ غیرپیش‌دستانه طبیعی است. آغاز زودتر بلندترین وظایف می‌تواند زمان تکمیل کل دسته را روی یک خوشه کوتاه کند، اما باید با تخمین زمان اجرا و سیاستی همراه شود که کارهای کوتاه را بی‌نهایت منتظر نگذارد (فصل ۱۲).

> **جمع‌بندی فصل**
>
> LJF به‌عنوان ابتکار Makespan برای بارهای معلوم روی منابع موازی و ابزار آموزشی توجیه‌پذیر است و به‌عنوان سیاست عمومی روی یک CPU توجیه ضعیفی دارد.

# فصل ۱۲: بهبودها و روش‌های جایگزین

تکنیک‌های زیر جزئی از الگوریتم کلاسیک LJF نیستند؛ روش‌های تکمیلی یا جایگزین‌اند که ضعف‌های آن را هدف می‌گیرند.

## ۱۲٫۱ Aging

Aging رتبهٔ فرایندهای منتظر را به‌تدریج بهبود می‌دهد. یک گونهٔ ساده از LJF بر اساس امتیاز s_i = BT_i + beta (t − AT_i) با beta > ۰ رتبه می‌دهد. کاری با زمان اجرای b که W واحد منتظر مانده، وقتی b + beta W > B_max شود از هر کار تازه‌واردی با BT حداکثر B_max جلو می‌افتد، یعنی حداکثر پس از (B_max − b)/beta واحد. پس هیچ کار آماده‌ای (جز انتظار برای اتمام کار جاری) بی‌نهایت منتظر نمی‌ماند و گرسنگی از بین می‌رود. با beta = ۰ همان LJF کلاسیک به دست می‌آید.

## ۱۲٫۲ Starvation Prevention

افزون بر Aging، می‌توان از گرسنگی با تضمین انتظار کراندار (ارتقای کاری که بیش از آستانه منتظر مانده)، رزرو سهمی از زمان CPU برای کارهای کوتاه یا اجرای کار بلند فقط وقتی طول صف زیر حدی است جلوگیری کرد.

## ۱۲٫۳ Hybrid Scheduling

سیاست ترکیبی LJF را برای رده‌های دسته‌ای و سیاستی دیگر را برای رده‌های تعاملی به کار می‌برد؛ مثلاً دو صف با Round Robin برای کارهای کوتاه و تعاملی و LJF برای صف دستهٔ پس‌زمینه، با تقسیم CPU میان صف‌ها.

## ۱۲٫۴ Dynamic Priority

اولویت پویا رتبه‌ها را هنگام اجرا از زمان انتظار، طول تخمینی یا CPU مصرف‌شده دوباره محاسبه می‌کند. Aging ساده‌ترین نمونه است؛ قاعدهٔ Highest Response Ratio Next با نسبت (W + BT)/BT نمونهٔ دیگری است که کارهای کوتاه را ترجیح می‌دهد ولی انتظار را هم پاداش می‌دهد [۲].

## ۱۲٫۵ ترکیب LJF و Priority Scheduling

ترکیب واژه‌نامه‌ای ابتدا بر اساس اولویت بیرونی رتبه می‌دهد و LJF را فقط میان فرایندهای هم‌اولویت به کار می‌برد. این کار کنترل مدیریتی (کارهای مهم اول) را حفظ می‌کند و مزیت Makespan را درون هر رده نگه می‌دارد.

## ۱۲٫۶ مدیریت بهتر پردازش‌های کوتاه

می‌توان از کارهای کوتاه با خط سریع جداگانه، سوئیچ به SJF وقتی طول صف از آستانه بیشتر شد یا شکستن کارهای بلند به قطعه‌های کوچک‌تر که با کارهای کوتاه درهم‌آمیزند محافظت کرد.

## ۱۲٫۷ سیاست‌های جایگزین برای افزایش عدالت

**جدول ۳۰.** روش‌های مکمل یا جایگزین LJF

| روش | ایده | مسئلهٔ هدف | هزینه |
|---|---|---|---|
| Aging | افزودن زمان انتظار به رتبه | گرسنگی | محاسبهٔ مجدد رتبه |
| ارتقای انتظار کراندار | ارتقا پس از حد مجاز | گرسنگی | نیاز به آستانه |
| صف‌های ترکیبی | سیاست متفاوت برای هر رده | تعاملی بودن | دو زمان‌بند |
| HRRN | رتبه BT/(BT + W) | عدالت | نیاز به BT |
| اولویت سپس LJF | اول اولویت، LJF برای تساوی | کنترل | تعیین اولویت |
| Round Robin | برش زمانی | پاسخ، عدالت | AWT بیشتر |

### جمع‌بندی فصل

Aging، صف‌های ترکیبی و اولویت پویا مشکل گرسنگی LJF را برطرف می‌کنند، به بهای فاصله‌گرفتن از الگوریتم کلاسیک.

# فصل ۱۳: جمع‌بندی و نتیجه‌گیری

## ۱۳٫۱ مرور مفاهیم اصلی

زمان‌بندی پردازنده فرایند بعدی را از صف آماده برمی‌گزیند. LJF بیشترین زمان اجرا را انتخاب می‌کند؛ شکل کلاسیک آن غیرپیش‌دستانه است، در حالی که خویشاوندان پیش‌دستانه‌اش LJF-PA و LRTF در کلید رتبه‌بندی و لحظات تصمیم تفاوت دارند. LJF و LRTF دو الگوریتم متفاوت‌اند.

## ۱۳٫۲ خلاصهٔ عملکرد الگوریتم و نتایج تحلیل عددی

**جدول ۳۱.** خلاصهٔ نتایج عددی (LJF مگر خلاف آن گفته شود، همراه با AWT الگوریتم SJF برای مرجع)

| بار کاری | n | ویژگی | AWT | ATAT | AWT در SJF |
|---|---|---|---|---|---|
| مثال ۱ | ۵ | همهٔ AT = ۰ | ۱۶٫۲۰ | ۲۲٫۶۰ | ۹٫۴۰ |
| مثال ۲ | ۵ | ورود متفاوت | ۷٫۲۰ | ۱۱٫۲۰ | ۴٫۴۰ |
| مثال ۳ | ۶ | BT یکسان | ۱۴٫۰۰ | ۱۹٫۸۳ | ۹٫۰۰ |
| مثال ۴ | ۱۰ | انباشت صف | ۲۶٫۴۰ | ۳۱٫۹۰ | ۱۳٫۴۰ |
| مثال ۵ | ۴ | LJF (NP) | ۶٫۲۵ | ۱۱٫۲۵ | ۴٫۲۵ |

در هر مثال LJF میانگین زمان انتظار بیشتری از SJF روی همان بار کاری دارد و فاصله وقتی بیشترین است که کارهای کوتاه فراوان در کنار کارهای بلند باشند.

## ۱۳٫۳ ارزیابی نقاط قوت و ضعف

نقاط قوت: سادگی، خدمت‌دهی زودهنگام به کارهای بلند، تحلیل‌پذیری و سودمندی به‌عنوان ابتکار Makespan. نقاط ضعف: میانگین زمان انتظار ضعیف، گرسنگی کارهای کوتاه، وابستگی به تخمین BT و نامناسب‌بودن برای استفادهٔ تعاملی.

## ۱۳٫۴ مقایسه با الگوریتم‌های دیگر

LJF از نظر میانگین انتظار و برگشت مغلوب SJF و SRTF، از نظر پاسخ و عدالت مغلوب Round Robin و از نظر کنترل مغلوب طرح‌های اولویت‌دار با Aging است. جایگاه آن توازن بار بلندترین‌اول است.

## ۱۳٫۵ محدودیت‌های الگوریتم و این مطالعه

تحلیل فرض می‌کند زمان‌های اجرا دقیقاً معلوم‌اند، هر فرایند یک CPU Burst دارد و هزینهٔ تعویض زمینه و I/O صفر است. مثال‌ها کوچک و قطعی‌اند. بارهای واقعی، اثر چندهسته‌ای و خطاهای پیش‌بینی BT خارج از دامنه‌اند.

## ۱۳٫۶ پیشنهاد برای تحقیقات آینده

- ارزیابی LJF با زمان‌های اجرای پیش‌بینی‌شده به‌جای دقیق و کمّی‌سازی اثر خطای تخمین.
- مطالعهٔ پارامترهای Aging که انتظار را کران‌دار می‌کنند و مزیت Makespan را حفظ می‌کنند.
- گسترش تحلیل به زمان‌بندهای چندپردازنده‌ای و بارهای دارای I/O Burst.
- مقایسهٔ LJF، LRTF و سیاست‌های عدالت‌محور روی ردهای بار کاری تجربی.

# فصل ۱۴: منابع و مراجع علمی

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Harlow: Pearson, 2017.

[3] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA: Pearson, 2015.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[5] W. E. Smith, “Various optimizers for single-stage production,” Naval Research Logistics Quarterly, vol. 3, no. 1–2, pp. 59–66, 1956. doi: 10.1002/nav.3800030106.

[6] R. L. Graham, E. L. Lawler, J. K. Lenstra and A. H. G. Rinnooy Kan, “Optimization and approximation in deterministic sequencing and scheduling: a survey,” Annals of Discrete Mathematics, vol. 5, pp. 287–326, 1979. doi: 10.1016/S0167-5060(08)70356-X.

[7] R. L. Graham, “Bounds on multiprocessor timing anomalies,” SIAM Journal on Applied Mathematics, vol. 17, no. 2, pp. 416–429, 1969. doi: 10.1137/0117039.

[8] M. L. Pinedo, Scheduling: Theory, Algorithms, and Systems, 5th ed. Cham: Springer, 2016. doi: 10.1007/978-3-319-26580-3.

[9] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

یادداشت: کتاب‌های بالا الگوریتم‌های کلاسیک (FCFS، SJF، SRTF، Round Robin، اولویت‌دار) و معیارهای ارزیابی این مقاله را ارائه می‌کنند؛ گزاره‌های اختصاصی دربارهٔ LJF و LRTF در این مقاله حاصل استنتاج نگارنده و راستی‌آزمایی عددی است، زیرا این سیاست‌ها فصل استاندارد آن کتاب‌ها نیستند. منابع به شیوهٔ بین‌المللی (IEEE) و به زبان اصلی آورده شده‌اند.